

SOFTWARE ENGINEERING

03

Introduction to Programming

Write your first line of code today — and understand exactly why it works.

DURATION	LEVEL	MODULE	UPDATED
15 Hours · 15 Sessions	Beginner	Module 3	April 2026

IN THIS MODULE

01	What is Programming?
02	Writing Your First C Program
03	Data Types and Variables
04	Operators and Expressions
05	Conditional Statements
06	Loops (For, While, Do-While)
07	Nested Loops & Pattern Printing
08	Functions
09	Arrays
10	Strings
11	Pointers
12	Structures
13	File Handling
14	Debugging & Best Practices
15	Mini Project: Student Productivity Tracker

SESSION

01

60 MIN · FUNDAMENTALS

What is Programming?

◆ Learning Objectives

- Define 'program' and 'programming language' in your own words
- Distinguish between how a compiler and an interpreter translate code
- Explain what an algorithm is and why it always comes before coding
- Read a basic flowchart and identify its four standard symbols
- Draw a simple flowchart for an everyday task

01 What Is a Program?

A program is a precise set of instructions written in a language a computer can understand and execute. Think of it like a recipe: each step is listed in the right order, and the computer follows them exactly. When you tap 'Order Now' on Swiggy, a program running on their servers checks the restaurant, calculates the delivery fee, and dispatches a partner — all in under a second.

CONCEPT — Program vs Programming Language

A programming language (C, Python, Java) is the grammar and vocabulary you use to write instructions. A program is the actual set of instructions you write using that language. C is the language; 'Hello, TOPS Technologies!' printed on screen is a program written in that language. The difference: one is the tool, the other is the output.

02 Compilers and Interpreters

Computers only understand binary (0s and 1s), so your human-readable code must first be translated before it can run. Two tools do this job. C uses a compiler — the entire file is read and translated into an executable all at once before running. Python uses an interpreter — it reads and runs code line by line, making testing easier but runtime slightly slower.

Compiler (C, Java)

Reads the entire source file first. Translates everything into a binary executable in one go. Reports all errors after scanning the full file. Produces fast-running output. GCC is the compiler you will use for C in this course.

Interpreter (Python, JavaScript)

Reads and executes code one line at a time. Stops and reports an error the moment it hits one. Easier for quick, interactive testing. Slightly slower at runtime because translation happens during execution.

03 Algorithms and Flowcharts

An algorithm is a finite, ordered set of steps that solves a problem. Every program begins as an algorithm — the code is simply a formal way to express it. A flowchart is the visual representation of an algorithm using standard shapes. Practising flowcharts before writing code builds the habit of thinking through a problem completely before touching the keyboard.

CONCEPT — Flowchart Symbols

Four shapes cover most flowcharts. Oval: Start or End of the program. Rectangle: Process or Action (e.g., 'Enter mobile number'). Diamond: Decision with a Yes/No branch (e.g., 'Is OTP correct?'). Arrow: Direction of flow between steps. Mastering these four is enough to map out any program before you write a single line of code.

TRY THIS LIVE

On paper, draw a flowchart for 'Ordering food on Swiggy.' Include: Start -> Open app -> Search restaurant -> Select dish -> Add to cart -> Payment successful? (Yes -> Show order confirmation / No -> Retry payment) -> End. Compare yours with a classmate and discuss any extra steps they included that you missed.

CASE STUDY — ISRO SAC Ahmedabad & Mangalyaan

Context — ISRO's Space Applications Centre (SAC) is headquartered in Ahmedabad. In 2013, SAC contributed critical software systems to India's Mars Orbiter Mission (Mangalyaan).

Challenge — The spacecraft would be over 400 million km from Earth at peak distance. A single software instruction error could destroy a Rs 450-crore mission with zero possibility of a remote fix.

What they did — ISRO engineers wrote exhaustive algorithms on paper and whiteboards before a single line of C code was committed. Every scenario — including thruster firing sequences and orbit insertion — was modelled as a flowchart and reviewed by multiple teams.

Result — Mangalyaan entered Mars orbit on its first attempt on 24 September 2014 — a feat no other space agency had achieved on its maiden mission.

Takeaway — Algorithm first, code second. The discipline that helped ISRO reach Mars is the same discipline you will practise every session of this module.

Source: ISRO Mission Report, Mars Orbiter Mission 2014, isro.gov.in

"Every app you use today was once just a flowchart on someone's whiteboard."

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Problem Solving through Programming in C — Lecture 1: Introduction to Programming (IIT Kharagpur, Prof. Anupam Basu)

Link — <https://nptel.ac.in/courses/106105085>

Always link the official channel — beware fan-made or re-upload copies.

WHERE TO GO NEXT

You now have the core mental model: programs are precise instructions, algorithms always come before code, and flowcharts make logic visible. Carry this forward into every session.

Explore deeper — Look up 'TIOBE Index 2024' and find where C ranks among all programming languages — it has been in the top 2 for over 40 years.

Try on your own — Pick any daily task (making chai, depositing cash at an ATM) and draw its complete flowchart with at least one decision diamond and two possible paths.

Connect the dots — Next session you will install GCC and write your first actual C program — the algorithm thinking you practised today becomes real code.

Going further — Read Chapter 1 of 'The Algorithm Design Manual' by Steven Skiena (a free excerpt is available online) to see how algorithm thinking is applied in industry.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a programming teacher for beginners. Explain the difference between a compiler and an interpreter in simple Hinglish. Use a Swiggy or Paytm example to make it relatable to Indian students. Keep it under 150 words.

SESSION

02

60 MIN · FUNDAMENTALS

Writing Your First C Program

◆ Learning Objectives

- Install GCC and verify it is working from the terminal
- Choose a suitable IDE (DevC++, VS Code, or Code::Blocks) and open a C file
- Identify and explain all four parts of a basic C program structure
- Write and save a 'Hello, TOPS Technologies!' program in C
- Compile and run a C program using GCC from the command line

01 Setting Up Your Environment

To write and run C programs you need two things: a compiler that translates your code into binary, and an editor where you write the code. The most widely used C compiler is GCC (GNU Compiler Collection) — it is free, open-source, and the same tool used by professional engineers at companies like Google, Intel, and Tata Elxsi. For the editor, DevC++ is the simplest to start with on Windows; VS Code is the most powerful and the one most industry teams use.

TRY THIS LIVE — Install and Verify GCC

Windows users: Download MinGW (which includes GCC) from mingw-w64.org and complete the installer. Then open Command Prompt and type: `gcc --version`. If you see a version number (e.g., `gcc 13.2.0`), your environment is ready. If you get 'not recognized', check that MinGW's bin folder has been added to your Windows PATH environment variable.

<code>gcc --version</code>	Confirms GCC is installed and prints the version number
<code>cd Desktop</code>	Navigate to your Desktop — a convenient place for your first projects
<code>mkdir tops_c</code>	Create a dedicated folder called <code>tops_c</code> for all your C files this module
<code>cd tops_c</code>	Move into that folder — all files you create will live here

02 Anatomy of a C Program

Every C program — no matter how large or complex — is built on the same four-part skeleton: a preprocessor directive to import libraries, the main function declaration, the body of instructions, and a return statement. Understanding what each part does before you type anything prevents the confusion that trips up most beginners in the first week.

CONCEPT — What Does `#include <stdio.h>` Mean?

`stdio.h` stands for Standard Input-Output Header. It is a built-in library file that gives your program access to functions like `printf()` (print to screen) and `scanf()` (take input from keyboard). The `#` symbol tells the compiler to process this line before anything else — that is why it is called a preprocessor directive. Without it, your program would not know what `printf()` is.

```
#include <stdio.h>
```

Preprocessor directive: loads the standard input-output library

```
int main() {
```

The main function: every C program starts executing here, always

```
    printf("Hello, TOPS Technologies!\n");
```

Prints text to the screen; `\n` moves the cursor to the next line

```
    return 0;
```

Returns 0 to the OS, signalling the program finished without errors

```
}
```

Closing brace: marks the end of the main function block

◆ Save this file as `hello.c` — the `.c` extension tells GCC it is a C source file

03 Compile and Run

Writing code is only step one. To turn your `.c` source file into a program the computer can run, you compile it with GCC. The compiler reads your file, checks for syntax errors, and produces an executable. If there are errors, GCC tells you the exact line number — fix them one by one from top to bottom, because one error near the top often triggers several false errors on the lines below it.

```
gcc hello.c -o hello
```

Compiles `hello.c` and produces an executable named 'hello'

```
./hello
```

Runs the compiled executable on Mac or Linux

```
hello.exe
```

Runs the compiled executable on Windows

```
gcc hello.c -o hello -Wall
```

`-Wall` enables all compiler warnings — always use this flag while learning

PRO TIP — Fix Errors Top-Down

When GCC shows 5 errors, do not try to fix all five at once. Fix only the first error listed and then recompile. Often that single fix makes 3-4 other 'errors' disappear — they were false positives caused by the first mistake. The top-down fix habit will save you hours across your entire programming career.

CASE STUDY — Zerodha's Low-Latency Trading Engine

Context — Zerodha, founded in Bengaluru in 2010, is India's largest stockbroker with over 1.2 crore active accounts. Their core order management system is built on C and C++.

Challenge — A stock trade must be routed and executed in microseconds. If the system is even 50ms slower than a competitor's, traders lose money. High-level interpreted languages are too slow for this requirement.

What they did — Zerodha's core order-routing engine was written in C — the same language you are learning today — because C compiles directly to machine code, giving it a speed advantage that no interpreted language can match at scale.

Result — Zerodha processes millions of orders daily with sub-millisecond execution times, making it cost-efficient enough to offer zero brokerage on equity delivery trades and disrupt the traditional broking industry.

Takeaway — The 'Hello, TOPS Technologies!' you write today runs in the same compiled C environment that powers crores of rupees of stock trades every hour at Zerodha.

Source: Zerodha Engineering Blog, zerodha.tech; SEBI Active Client Data, 2024

“The program you compile in the next 30 minutes runs in the same language that powers India's fastest trading back-ends.”

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Problem Solving through Programming in C — Lecture 2: Writing and Executing a Simple C Program (IIT Kharagpur, Prof. Anupam Basu)

Link — <https://nptel.ac.in/courses/106105085>

WHERE TO GO NEXT

You have written, compiled, and run your first C program. That is a real milestone. The four-part structure you learned today is the container for every program you will build this module.

Explore deeper — Search 'cprogramming.com history of C' — understanding why C was created by Dennis Ritchie in 1972 makes its design decisions make far more sense.

Try on your own — Modify your hello.c to print your own name, your city, and your course name — each on a separate line using three separate printf() calls.

Connect the dots — Next session you will store data — a student's name, age, and marks — inside your program by learning variables and data types.

Going further — Look up 'GCC optimisation flags -O0 -O1 -O2 -O3' to understand how the compiler can

automatically make your code run faster at compile time.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a C programming teacher. Explain step by step what happens when I type 'gcc hello.c -o hello' and then run './hello' — in simple Hinglish. Use the analogy of a translator converting a Gujarati letter into English so a foreign colleague can read it. Keep it under 150 words.

SESSION

03

60 MIN · FUNDAMENTALS

Data Types and Variables

◆ Learning Objectives

- Explain what a variable is in terms of named memory storage
- Declare variables using the four fundamental C data types: int, float, char, and double
- Apply all variable naming rules and avoid the most common mistakes
- Distinguish between a constant and a literal, and use both correctly in a program
- Write a C program that stores and displays a student's name, age, marks, and grade

01 Variables — Named Boxes in Memory

A variable is a named location in the computer's RAM where you store a value your program can read and change. Think of it like a labelled box in a storage room: the label is the variable name, and whatever you put inside is the value. In C, before you can use any variable, you must declare it — telling the compiler what type of data it will hold so the right amount of memory is reserved.

CONCEPT — Variable Declaration Syntax

The pattern is: `data_type variable_name;` — for example, `int age;` declares a variable named 'age' that stores whole numbers. You can also declare and initialise in one step: `int age = 20;` Always initialise variables before using them — an uninitialised variable holds whatever random 'garbage' bytes happen to be in that memory location, which causes unpredictable bugs.

02 The Four Fundamental Data Types

C is a strongly typed language — every variable must have a declared type, and the compiler enforces it. The type tells the compiler two things: how many bytes of memory to reserve, and how to interpret those stored bytes. The four most-used basic types are int, float, char, and double. Choosing the wrong type — like storing a student's percentage as an int — causes silent errors that are painful to debug.

Data Type	What It Stores	Memory Size	Example Values
int	Whole numbers (no decimal point)	4 bytes	20, -5, 100, 0
float	Decimal numbers (single precision)	4 bytes	9.5, 3.14, -0.5
double	Decimal numbers (higher precision)	8 bytes	91.6666667, 3.14159265

Data Type	What It Stores	Memory Size	Example Values
char	A single character or symbol	1 byte	'A', 'z', '3', '@'

```
#include <stdio.h>
int main() {
    int age = 20;

    float marks = 87.5;

    char grade = 'A';
    double percentage = 91.666667;

    printf("Age: %d\n", age);
    printf("Marks: %.2f\n", marks);
    printf("Grade: %c\n", grade);
    printf("Percentage: %.4f\n", percentage);
    return 0;
}
```

Load the standard input-output library
Program entry point
Whole number: stores the student's age without decimals
Decimal number: stores marks that can have a fractional part
Single character: note single quotes, not double quotes
Higher precision decimal: better for calculated percentages
%d is the format specifier for int values
%.2f prints a float rounded to 2 decimal places
%c is the format specifier for char values
%.4f prints double with 4 decimal places
Signal successful execution to the operating system
End of main function

03 Variable Naming Rules

C enforces strict naming rules — break them and your program will not compile. A variable name may contain letters (a-z, A-Z), digits (0-9), and underscores (_). It must start with a letter or underscore, never a digit. Spaces are not allowed. C is case-sensitive: 'marks' and 'Marks' are two completely different variables. The widely accepted style in C is snake_case — words separated by underscores, e.g., student_marks.

WARNING — Common Naming Mistakes

These names will cause a compilation error: 2marks (starts with a digit), student marks (space inside the name), int (reserved keyword). These names compile but create bad habits: x, a, temp (meaningless), sm (cryptic abbreviation). Use descriptive names: student_age, monthly_fee, passing_marks. The computer does not care — but your teammates and future self will.

04 Constants and Literals

A literal is a fixed value written directly in code, like 35 or 3.14. A constant is a named literal — a name given to a fixed value so it can be referenced meaningfully throughout the code. Using named constants instead of raw literals makes your code readable and easy to maintain. If the passing mark changes from 35 to 40, you update one constant and every calculation in the program updates automatically.

CONCEPT — #define vs const

#define PASSING_MARKS 35 is a preprocessor macro: before compiling, the compiler replaces every occurrence of PASSING_MARKS with the literal 35. It has no type. const int PASSING_MARKS = 35; is a typed, memory-allocated constant — the compiler enforces its type. In modern C code, prefer const int; use #define only when a raw text substitution is specifically needed.

TRY THIS LIVE

Extend the code example above: add char name[] = "Rahul"; at the top and print it with printf("Name: %s\n", name); — %s is the format specifier for a string (an array of characters). Run the program and confirm all five fields print correctly: name, age, marks, grade, and percentage. Then swap the values for your own details.

CASE STUDY — GSTN: Storing India's GST Data

Context — The GST Network (GSTN) is the technology backbone of India's Goods and Services Tax system. It processes tax returns for over 1.4 crore registered businesses, including thousands of firms across Gujarat's textile, pharma, and manufacturing sectors.

Challenge — Every GST transaction must store several different kinds of information at once: a 15-character GSTIN code, a taxable amount in rupees and paise, a tax rate as a percentage, a transaction date, and a taxpayer category — each requiring a different type of storage to be accurate and efficient.

What they did — GSTN's database architects used precisely typed columns for every field: VARCHAR for GSTIN (a text string), DECIMAL for rupee amounts (to avoid float rounding errors on money), TINYINT for tax slabs (0, 5, 12, 18, 28), and DATE for filing timestamps — the exact same data-type decisions you are practising in C today.

Result — GSTN processes over 4 crore return filings per month with sub-second validation, because correct data types eliminate type-mismatch errors and minimise wasted storage at national scale.

Takeaway — Choosing int vs float vs char is not a beginner classroom exercise — it is the same architectural decision that GSTN made for a system handling lakhs of crores of rupees in tax data.

Source: GSTN Technical Architecture Overview, gstn.org.in; GSTN Press Release 2023

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Problem Solving through Programming in C — Lecture 4: Variables and Data Types (IIT Kharagpur, Prof. Anupam Basu)

Link — <https://nptel.ac.in/courses/106105085>

WHERE TO GO NEXT

You can now declare variables of the correct type, name them following C rules, and use constants for fixed

values. These three skills are the foundation every program you write this module will build on.

Explore deeper — Search 'C data types size on 32-bit vs 64-bit systems' — understanding why int is 2 bytes on some older compilers and 4 on modern ones explains a class of real-world bugs.

Try on your own — Write a new program that stores a product's name (char array), price in rupees (float), quantity in stock (int), and GST rate (double) — then prints a formatted product summary.

Connect the dots — Next session covers operators — you will perform arithmetic, comparisons, and logical checks on the variables you declared today.

Going further — Search 'Ariane 5 rocket integer overflow 1996' — a real-world disaster caused by storing a 64-bit float value into a 16-bit integer. It is the most famous data-type bug in history.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a C programming tutor for beginners. Explain the difference between int, float, char, and double in simple Hinglish using Indian everyday examples — like storing a student's roll number, CGPA, grade letter, and exact percentage. Keep it under 150 words.

SESSION

04

60 MIN · FUNDAMENTALS

Operators and Expressions

◆ Learning Objectives

- Identify and apply the five arithmetic operators in C expressions
- Use relational operators to compare two values and produce a true (1) or false (0) result
- Combine conditions using the logical operators &&, ||, and !
- Differentiate between prefix (++i) and postfix (i++) increment and decrement behaviour
- Apply operator precedence rules and use parentheses to control expression evaluation order

01 Arithmetic Operators

An operator is a symbol that tells the compiler to perform a specific operation on one or more values. Arithmetic operators in C work like the maths you already know — with one important exception. When both operands are integers, division also returns an integer and silently drops the decimal. This catches every beginner by surprise: in C, $10 / 3$ gives 3, not 3.33. The remainder left behind is what the modulus operator (%) captures.

Operator	Name	Example (a=10, b=3)	Result
+	Addition	$a + b$	13
-	Subtraction	$a - b$	7
*	Multiplication	$a * b$	30
/	Division	a / b	3 (decimal dropped — integer division)
%	Modulus (remainder)	$a \% b$	1 ($10 = 3 \times 3 + 1$)

CONCEPT — Integer Division and Modulus

$10 / 3 = 3$ in C because both operands are int — the fractional part is discarded without rounding. To get 3.333333, make at least one operand a float: $10.0 / 3$ works. Modulus (%) only works with integers and returns the remainder: $10 \% 3 = 1$. A classic use: $n \% 2 == 0$ checks if n is even — you will use this in every loop session from here.

```
#include <stdio.h>
int main() {
    int a = 10, b = 3;
    printf("Addition: %d\n", a + b);
```

Load standard I/O library

Program entry point

Declare two integer variables in one statement

Prints 13

```
printf("Subtraction:    %d\n", a - b);
printf("Multiplication: %d\n", a * b);
printf("Division:       %d\n", a / b);
printf("Modulus:        %d\n", a % b);
printf("Float div:      %.2f\n", 10.0 / b);
return 0;
```

Prints 7
Prints 30
Prints 3 — decimal is dropped (integer division)
Prints 1 — remainder after 10 ÷ 3
Prints 3.33 — one float operand forces decimal division
Signal successful completion
End of main function

02 Relational and Logical Operators

Relational operators compare two values and always return either 1 (true) or 0 (false) in C — there is no separate boolean type in C89/C90. You will use these constantly: checking whether a student has passed, whether a balance is sufficient, or whether a value falls within range. Logical operators combine multiple relational expressions into a single true/false result, letting you express compound conditions in one line.

Operator	Meaning	Example	Result
==	Equal to	5 == 5	1 (true)
!=	Not equal to	5 != 3	1 (true)
>	Greater than	5 > 3	1 (true)
<	Less than	3 < 5	1 (true)
>=	Greater than or equal	5 >= 5	1 (true)
<=	Less than or equal	2 <= 5	1 (true)

CONCEPT — Logical Operators: &&, ||, and !

&& (AND): the result is 1 only if BOTH conditions are true. marks >= 35 && attendance >= 75 — a student passes only if both are met. || (OR): the result is 1 if AT LEAST ONE condition is true. isHoliday || isWeekend — the office is closed if either is true. ! (NOT): flips the result — !(marks >= 35) is identical to marks < 35. Critical warning: never write = where you mean == inside a condition. if (marks = 35) assigns 35 to marks and is always true — a silent and dangerous bug.

TRY THIS LIVE

Add these four printf lines to your program and predict the output before running. printf("%d\n", 10 > 3); — should print 1. printf("%d\n", 10 == 3); — should print 0. printf("%d\n", (10 > 3) && (2 < 5)); — should print 1. printf("%d\n", (10 < 3) || (2 < 5)); — should print 1. Confirm each result matches your prediction before moving

on.

03 Increment, Decrement, and Operator Precedence

The increment operator (++) adds 1 to a variable; the decrement operator (--) subtracts 1. You will use them in almost every loop you write. The placement matters: prefix (++) modifies the variable first, then uses the updated value in the expression. Postfix (i++) uses the current value first, then modifies the variable. In a standalone statement like i++; the difference has no visible effect — it only matters when the operator is embedded inside a larger expression.

Prefix: ++i / --i

Modify FIRST, then use. `int i = 5; printf("%d", ++i);` prints 6 — i is incremented to 6 before printf sees it. After this line, i = 6.

Postfix: i++ / i--

Use FIRST, then modify. `int i = 5; printf("%d", i++);` prints 5 — printf uses the original value, then i is incremented to 6. After this line, i = 6.

```
int i = 5;
printf("%d\n", ++i);
printf("%d\n", i++);
printf("%d\n", i);
printf("%d\n", 2 + 3 * 4);
printf("%d\n", (2 + 3) * 4);
```

Start with i = 5
PREFIX: i becomes 6 first — prints 6
POSTFIX: prints 6 (current value), then i becomes 7
Confirm: i is now 7
Precedence: * before + — prints 14, not 20
Parentheses force + first — prints 20

Priority	Operators	Rule	Example
1 — Highest	()	Parentheses are always evaluated first	(2 + 3) * 4 = 20
2	*, /, %	Multiplication, Division, Modulus	2 + 3 * 4 = 14
3	+, -	Addition, Subtraction	10 - 2 + 1 = 9
4	>, <, >=, <=	Relational comparisons	5 + 3 > 7 -> 1
5 — Lowest	==, !=	Equality and inequality checks	8 == 8 -> 1

CASE STUDY — NSE Circuit Breaker: Operators Protecting a Live Market

Context — The National Stock Exchange (NSE), Mumbai, operates India's largest equity market. Every trading day, crores of rupees of transactions flow through their automated surveillance system in real time.

Challenge — When a stock or index moves too rapidly — rising or falling by 10% or 20% in a single session — the exchange must detect the breach instantly and halt trading to protect investors from panic-driven losses.

What they did — NSE's surveillance engine evaluates relational operators continuously: if (price_change_pct >= 10.0) — lower circuit triggered, trading paused for 15 minutes. if (price_change_pct >= 20.0) — stock halted for the day. Logical operators combine index-level and stock-level thresholds using && to determine whether a market-wide halt is needed.

Result — During the COVID-19 market crash of March 2020, circuit breakers triggered four times in a single month — protecting retail investors from runaway sell-offs through exactly these relational and logical operator comparisons.

Takeaway — The >, >=, and && operators you practised today are the same ones protecting India's Rs 200-lakh-crore stock market from automated panic crashes.

Source: NSE Surveillance Circular, nseindia.com; SEBI Circuit Breaker Mechanism, sebi.gov.in

“Every condition a computer ever checks — from ATM PINs to stock circuit breakers — comes down to a relational operator returning 1 or 0.”

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Problem Solving through Programming in C — Lecture 5: Operators and Expressions (IIT Kharagpur, Prof. Anupam Basu)

Link — <https://nptel.ac.in/courses/106105085>

WHERE TO GO NEXT

You can now write expressions using all four operator types and predict the result. This computational power is the engine inside every decision your programs will ever make.

Explore deeper — Search 'C operator precedence table cppreference' and bookmark it — even senior developers refer to this table. Look at all 15 precedence levels and notice where assignment (=) sits.

Try on your own — Extend your calculator: use scanf() to read two float values from the user, then print all five arithmetic results — including integer vs float division side by side to see the difference.

Connect the dots — Next session, conditional statements: you will wrap these relational and logical expressions inside if and switch blocks to make your programs take real decisions based on real input.

Going further — Research 'integer overflow in C' and what happens when you increment an int past 2,147,483,647. This is a real class of production bugs — including the famous Ariane 5 rocket failure of 1996.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a C programming teacher. Explain the difference between prefix (++i) and postfix (i++) increment in simple Hinglish. Use the analogy of a tea stall owner who either counts the cup before pouring or after pouring. Keep it under 150 words.

SESSION

05

60 MIN · FUNDAMENTALS

Conditional Statements

◆ Learning Objectives

- Write if and if-else statements to branch program execution based on a condition
- Build an else-if ladder to handle multiple mutually exclusive conditions cleanly
- Use nested if statements to express compound multi-level decisions
- Implement a switch-case block for clean, readable menu-driven programs
- Build a 'Study Mood Bot' that reads user input and responds with a motivational message

01 The if Statement

Until now, every program you wrote ran every line from top to bottom, every time. Conditional statements change that — they let a program choose which lines to run based on whether a condition is true or false. This is the foundation of all decision-making in software. The if statement is the simplest form: if the condition inside the parentheses evaluates to 1 (true), the block inside the curly braces runs; if it evaluates to 0 (false), that block is skipped entirely.

CONCEPT — if Statement Syntax

if (condition) { // runs only when condition is true }. The condition is any expression that resolves to 0 or non-zero — relational and logical expressions from Session 04 are the most common. The curly braces { } define the block. For a single-line body you can omit the braces, but always include them — it prevents a common class of bugs when you add a second line later.

<code>#include <stdio.h></code>	Load standard I/O library
<code>int main() {</code>	Program entry point
<code> int marks = 72;</code>	A student's marks stored as an integer
<code> if (marks >= 35) {</code>	Condition: is 72 >= 35? Yes (1) — block runs
<code> printf("Result: PASS\n");</code>	This line runs because the condition was true
<code> }</code>	End of if block
<code> printf("Check complete.\n");</code>	This line ALWAYS runs — it is outside the if block
<code> return 0;</code>	Signal successful completion
<code>}</code>	End of main function

02 if-else and the else-if Ladder

An if statement only handles the 'yes' case. Adding else handles the 'no' case — so every outcome is covered. When there are more than two possible outcomes, chain multiple else-if blocks into a ladder. C evaluates conditions from top to bottom and stops the moment one is true, executing only that block. This makes the else-if ladder efficient and

readable for grading, routing, and classification logic.

<code>#include <stdio.h></code>	Load standard I/O library
<code>int main() {</code>	Program entry point
<code> int marks = 75;</code>	Change this value to test each branch
<code> if (marks >= 90) {</code>	First condition checked — 75 >= 90 is false, skip
<code> printf("Grade: A\n");</code>	Runs only for 90 and above
<code> } else if (marks >= 75) {</code>	Second check — 75 >= 75 is true, this block runs
<code> printf("Grade: B\n");</code>	Prints Grade: B — remaining else-if blocks are skipped
<code> } else if (marks >= 60) {</code>	Not reached — C already found a true condition above
<code> printf("Grade: C\n");</code>	Runs for 60–74
<code> } else if (marks >= 35) {</code>	Runs for 35–59
<code> printf("Grade: D – Pass\n");</code>	Minimum passing grade
<code> } else {</code>	Catches everything below 35
<code> printf("Grade: F – Fail\n");</code>	No condition was true above — this is the fallback
<code> }</code>	End of the else-if ladder
<code> return 0;</code>	Signal successful completion
<code>}</code>	End of main function

PRO TIP — else-if vs Separate if Statements

Writing three separate if statements one after another means all three are checked every time — even after one already matched. An else-if ladder stops checking the moment it finds the first true condition. For mutually exclusive outcomes (a student can only have one grade), always use else-if — it is more correct and more efficient. Use separate if statements only when multiple conditions can independently be true at the same time.

03 Nested if

A nested if is an if statement placed inside another if block. Use it when a second condition only makes sense to check after a first condition is already confirmed true. For example: only check a student's attendance if their marks already confirm they are passing — checking attendance on a failing student is irrelevant. Nesting lets you model these compound, multi-level decisions clearly.

<code>int marks = 60, attendance = 80;</code>	Two separate conditions to check
<code>if (marks >= 35) {</code>	Outer if: check marks first
<code> if (attendance >= 75) {</code>	Inner if: only reached if marks condition is true
<code> printf("Eligible for exam: YES\n");</code>	Both conditions met
<code> } else {</code>	Marks pass but attendance is low
<code>printf("Low attendance – not eligible\n");</code>	
<code> }</code>	Inner else
<code>}</code>	End of inner if-else

```

} else {
    printf("Marks below passing – not eligible\n");
}

```

Outer else: marks below 35, attendance check is irrelevant

Outer else

End of outer if-else

WARNING — Avoid Deep Nesting

Do not nest if statements more than 2–3 levels deep. Code that looks like a staircase of indentation — if inside if inside if inside if — is called the 'pyramid of doom' and is a sign the logic needs to be redesigned. Before nesting deeper, ask: can I combine these conditions with && instead? Often, if (marks >= 35 && attendance >= 75) replaces two nested levels with one clean line.

04 Switch-Case

When your program needs to take one of several actions based on the exact integer value of a single variable, switch-case is cleaner and more readable than an else-if ladder. Think of switch as a multi-door corridor: based on the value, C jumps directly to the matching door (case) and starts executing from there. Without a break statement, execution falls through to the next case — which is almost always a bug.

CONCEPT — switch Syntax and the break Statement

switch (variable) { case value1: ... break; case value2: ... break; default: ... }. The variable must be an integer or char — it cannot be a float or a range. Each case must end with break; to stop execution after that block. The default: block (optional but recommended) runs when no case matches, exactly like the final else in an else-if ladder.

CONCEPT — Taking User Input with scanf

To read an integer from the keyboard: int mood; scanf("%d", &mood); The & symbol is the address-of operator — it tells scanf where in memory to store the value the user types. Forgetting & is one of the most common beginner bugs: scanf("%d", mood) compiles but behaves incorrectly at runtime. Always use & with scanf for basic variable types.

```

#include <stdio.h>
int main() {
    int mood;
    printf("How are you feeling? ");
    printf("(1=Motivated 2=Tired 3=Stressed): ");
}

```

Load standard I/O library

Program entry point

Variable to store the user's menu choice

Prompt the user

Show available options

<code>scanf("%d", &mood);</code>	Read an integer from the keyboard into 'mood'
<code>switch (mood) {</code>	Jump to the case that matches the value of mood
<code>case 1:</code>	Runs only when mood == 1
<code>printf("You are on fire! Keep this energy.\n");</code>	
Motivational response for Motivated	
<code>break;</code>	Exit the switch — without this, case 2 would also run
<code>case 2:</code>	Runs only when mood == 2
<code>printf("Take a 10-min break and come back stronger.\n");</code>	
Response for Tired	
<code>break;</code>	Exit the switch
<code>case 3:</code>	Runs only when mood == 3
<code>printf("Break it into 5-min pieces. Start now.\n");</code>	
Response for Stressed	
<code>break;</code>	Exit the switch
<code>default:</code>	Runs when mood is anything other than 1, 2, or 3
<code>printf("Invalid choice. Enter 1, 2, or 3.\n");</code>	
Always include a default to handle unexpected input	
<code>}</code>	End of switch block
<code>return 0;</code>	Signal successful completion
<code>}</code>	End of main function

TRY THIS LIVE — Extend the Study Mood Bot

Compile and run the Study Mood Bot above. Once it works, add a fourth option: case 4 — print 'Distracted? Put your phone away for 25 minutes. One focused session changes the day.' Then test what happens when you deliberately remove one break statement — watch the fall-through behaviour and understand why break is non-negotiable.

CASE STUDY — IRCTC Ticket Booking: Conditional Logic at Scale

Context — IRCTC (Indian Railway Catering and Tourism Corporation) has managed online railway ticket bookings since 2002. It handles over 8 lakh ticket bookings every single day across all classes and quotas.

Challenge — For every booking request, the system must instantly determine which ticket status to assign — Confirmed (CNF), Reservation Against Cancellation (RAC), Waitlisted (WL), or Rejected — based on seat availability, travel class, quota type, and whether Tatkal is active.

What they did — The core allocation engine uses a nested conditional chain: if (seats_available > 0) -> CNF; else if (rac_slots > 0) -> RAC; else if (wl_count < max_waitlist) -> WL; else -> Rejected. Tatkal availability adds another outer if layer checked first, with its own separate quota logic inside.

Result — IRCTC processes 8 lakh+ bookings daily with real-time status decisions, serving crores of passengers across India — all driven by exactly the conditional structures you wrote today.

Takeaway — Every time your IRCTC booking returns CNF, RAC, or WL, a chain of if-else and nested if conditions — identical to the ones in this session — ran in milliseconds to make that call.

Source: IRCTC Annual Report 2022-23; Ministry of Railways Booking Statistics, indianrailways.gov.in

“Every 'Confirmed' or 'Waitlisted' you have ever seen on IRCTC is the output of an if-else chain running in under a second.”

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Problem Solving through Programming in C — Lecture 7: Conditional Statements (IIT Kharagpur, Prof. Anupam Basu)

Link — <https://nptel.ac.in/courses/106105085>

WHERE TO GO NEXT

Your programs can now take different paths based on real input. That is the moment code stops being a sequence of instructions and starts being genuinely intelligent behaviour.

Explore deeper — Search 'dangling else problem in C' — a classic ambiguity that shows why indentation alone is not reliable and why curly braces always matter.

Try on your own — Extend the grade classifier to also print the marks range for each grade after the grade letter — for example: 'Grade: B (75 – 89 marks)'. Then add a check that rejects invalid input (marks < 0 or marks > 100).

Connect the dots — Next session: loops — instead of checking one student's marks, you will check many, automatically, using for and while loops that repeat your conditional logic across entire arrays.

Going further — Look up 'switch fall-through as a feature in C' — there are rare, legitimate uses for intentionally omitting break. Understanding them shows you have truly mastered switch-case.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a C programming teacher. Explain the difference between an else-if ladder and a switch-case statement in simple Hinglish. Use the example of an IRCTC ticket counter assigning seat types (Confirmed, RAC, Waitlist). When should a developer choose one over the other? Keep it under 150 words.

SESSION

06

60 MIN · FUNDAMENTALS

Loops (For, While, Do-While)

◆ Learning Objectives

- Explain the difference between entry-controlled and exit-controlled loops
- Write a for loop correctly using its three-part header: initialisation, condition, update
- Use a while loop for condition-based repetition where iteration count is unknown
- Implement a do-while loop for a menu-driven program that must run at least once
- Build a countdown timer program using a for loop

01 Why Loops? The Power of Repetition

Without loops, printing the numbers 1 to 100 would require 100 separate `printf()` lines. With a loop, you write the instruction once and tell C how many times to repeat it. This is not just a shortcut — it is the mechanism that makes computing powerful. Every search result, every social media feed, every transaction history report you have ever seen was generated by a program iterating through data with a loop.

CONCEPT — Entry-Controlled vs Exit-Controlled Loops

Entry-controlled loops (for, while) check their condition **BEFORE** executing the body. If the condition is false from the start, the body never runs even once. Exit-controlled loops (do-while) execute the body **FIRST** and check the condition **AFTER**. This guarantees the body runs at least once — essential for menus and input validation where you always need one execution before deciding to repeat.

Loop Type	Control	Use When	Iteration Count
for	Entry-controlled	Number of iterations is known in advance	Known (e.g., count 1 to 10)
while	Entry-controlled	Loop while a condition holds, count unknown	Unknown (e.g., read until user types 0)
do-while	Exit-controlled	Body must execute at least once before checking (e.g., menus)	At least 1, then condition-based

02 The for Loop

The for loop packs all three loop-management tasks — initialisation, condition check, and update — into a single header line. This makes it the most readable choice when you know exactly how many times you need to iterate. The three parts are separated by semicolons and each plays a specific role: the first runs once before the loop starts, the

second is checked before every iteration, and the third runs after every iteration.

CONCEPT — for Loop Anatomy

for (initialisation; condition; update) { body }. Initialisation: sets the starting value — runs exactly once. Condition: checked before each iteration — loop runs only while this is true. Update: executes after each iteration — usually increments or decrements the counter. Any of the three parts can be omitted, but the semicolons must remain. Omitting the condition (for ;;) creates an infinite loop.

<code>#include <stdio.h></code>	Load standard I/O library
<code>int main() {</code>	Program entry point
<code> int i;</code>	Loop counter variable
<code> for (i = 10; i >= 1; i--) {</code>	Init: i=10. Condition: run while i>=1. Update: decrement i after each run
<code> printf("%d...\n", i);</code>	Body: prints 10... 9... 8... down to 1...
<code> }</code>	When i reaches 0 the condition i>=1 is false — loop ends
<code> printf("Launch!\n");</code>	Runs once after the loop completes — outside the loop body
<code> return 0;</code>	Signal successful completion
<code>}</code>	End of main function

03 The while Loop

The while loop is the right choice when you do not know in advance how many iterations are needed — you just want to keep going as long as a condition is true. Unlike for, the counter is declared and initialised outside the loop. This also means it is easy to forget the update step inside the body, which creates an infinite loop — the most common beginner mistake with while. Always double-check that something inside the body will eventually make the condition false.

<code>int i = 1;</code>	Initialise counter BEFORE the loop — unlike for, this is outside
<code>while (i <= 5) {</code>	Entry-controlled: condition is checked before every iteration
<code> printf("%d\n", i);</code>	Body: print current value of i
<code> i++;</code>	UPDATE — this line is critical. Forget it and the loop never ends
<code>}</code>	Loop exits when i > 5 (i becomes 6 and condition fails)

WARNING — The Infinite Loop Trap

while (1) { printf("hello\n"); } runs forever — the condition is always true and there is no exit. If your program seems stuck or your terminal is printing endlessly, press Ctrl + C to force-stop it. When writing a while loop, always verify before running: is there a statement inside the body that will eventually make the condition false? If not, you have an infinite loop.

04 The do-while Loop

The do-while loop flips the order: the body executes first, and the condition is checked at the bottom. This guarantees at least one execution regardless of the condition's initial value. The most practical use is menu-driven programs — you always want to display the menu at least once before deciding whether to loop back and show it again based on the user's choice.

CONCEPT — do-while Syntax

`do { body } while (condition);` — note the semicolon after the closing parenthesis; forgetting it is a common syntax error. The body runs first unconditionally. Then the condition is checked — if true, the body runs again; if false, the loop exits. This single structural difference from while is what makes do-while the correct choice for any program that must present output before asking whether to continue.

```
int choice;                                Variable to store the user's menu selection
do {                                        Body executes BEFORE the condition is checked —
                                           menu always shows at least once

printf("\n1. Study Now  2. Take a Break  3. Exit\n");
    Display the menu
    printf("Enter choice: ");                Prompt for input
    scanf("%d", &choice);                    Read the user's integer choice
if (choice == 1)    printf("Opening notes...\n");
    Handle option 1
else if (choice == 2) printf("10-minute break started.\n");
    Handle option 2
else if (choice != 3) printf("Invalid. Enter 1, 2, or 3.\n");
    Reject anything that is not 1, 2, or 3
} while (choice != 3);                      Exit-control: keep looping until user selects 3
printf("Session ended. Good work!\n");      Runs once after the loop exits on choice == 3
```

TRY THIS LIVE

Combine both examples: wrap the countdown timer inside the do-while menu as option 1. When the user selects 1, the program counts down from 10 to 1 and prints 'Study session started!' When they select 2, it prints the break message. When they select 3, the program exits. This single program uses all three loop types: for (countdown), and do-while (menu).

CASE STUDY — Amul GCMMF: Loops Processing 36 Lakh Farmers Daily

Context — The Gujarat Co-operative Milk Marketing Federation (GCMMF) — Amul — is headquartered in Anand, Gujarat. It connects 36 lakh farmer-members across 18,600+ village dairy cooperative societies to markets worldwide.

Challenge — Every evening, milk collection data arrives from all 18,600+ village societies. Each record contains individual farmer IDs, quantities collected, and fat content readings. The system must compute daily payment totals, fat-based bonuses, and running monthly summaries — for millions of individual submissions — overnight.

What they did — Amul's automated data processing system uses nested iteration: a loop across all 36 districts, within it a loop across each cooperative society, and within that a loop through each farmer's submission for the day. Each pass accumulates totals and triggers payment calculations — exactly the repetition pattern you practised today.

Result — The system processes and generates accurate payment data for all 36 lakh farmer-members daily, underpinning Amul's Rs 72,000-crore annual revenue and making it India's largest food brand by sales.

Takeaway — The for loop counting down from 10 that you wrote today and the loops computing millions of dairy payments in Anand tonight are the same construct — one instruction, repeated precisely.

Source: GCMMF Annual Report 2022-23; Amul official data, amul.com

“A loop does not repeat your work — it multiplies it. One instruction, executed a million times, is still one instruction.”

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Problem Solving through Programming in C — Lecture 9: Loops — for, while, do-while (IIT Kharagpur, Prof. Anupam Basu)

Link — <https://nptel.ac.in/courses/106105085>

WHERE TO GO NEXT

You can now make your program repeat any block of instructions with all three loop types. Combined with conditionals from Session 05, your programs can now handle real-world, variable-length tasks.

Explore deeper — Search 'break and continue in C loops' — these two keywords let you exit a loop early or skip the current iteration, and you will need them in Session 09 (Arrays).

Try on your own — Rewrite the countdown timer using a while loop, and then again using a do-while loop. Compare all three versions side by side — same output, different structure. This comparison builds strong loop intuition.

Connect the dots — Next session: nested loops — placing a loop inside another loop to generate 2D patterns. Every pattern you draw is practice for working with 2D arrays and grids.

Going further — Search 'loop unrolling optimisation in C' to understand how GCC's -O2 flag automatically replaces short loops with repeated inline code for speed — a glimpse into how compilers think.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a C programming teacher. Explain the difference between for, while, and do-while loops in simple Hinglish. Use the example of a Swiggy delivery rider who keeps delivering until their shift ends (while), makes exactly 5 deliveries per slot (for), and always at least picks up the first order before checking if the route is viable (do-while). Keep it under 150 words.

SESSION

07

60 MIN · FUNDAMENTALS

Nested Loops & Pattern Printing

◆ Learning Objectives

- Explain how nested loops execute: inner loop completes fully for each single outer iteration
- Trace a nested loop manually on paper to predict its output before running
- Print a rectangle of stars using two nested for loops
- Print a right-angle triangle pattern by making the inner loop bound depend on the outer counter
- Print a centred star pyramid using three inner loops (spaces, stars, newline)

01 How Nested Loops Work

A nested loop is a loop placed inside the body of another loop. The outer loop controls rows and the inner loop controls columns — this is the mental model that makes every pattern, grid, and 2D data structure in programming legible. The key rule: for every single iteration of the outer loop, the inner loop runs to completion. If the outer runs 5 times and the inner runs 5 times, the inner body executes $5 \times 5 = 25$ times total.

CONCEPT — Execution Order in Nested Loops

Outer $i=1 \rightarrow$ inner j runs 1,2,3,4,5 $\rightarrow$ newline. Outer $i=2 \rightarrow$ inner j runs 1,2,3,4,5 $\rightarrow$ newline. And so on until the outer condition fails. The inner loop variable (j) resets to its initial value at the start of every outer iteration. Never use the same variable name for both loops — using i for both creates a naming conflict that produces wrong results.

```
#include <stdio.h>
int main() {
    int i, j;
    for (i = 1; i <= 5; i++) {
        for (j = 1; j <= 5; j++) {
            printf("* ");
        }
        printf("\n");
    }
    return 0;
}
```

Load standard I/O library
Program entry point
Declare both loop counters — i for rows, j for columns
Outer loop: runs 5 times — one iteration per row
Inner loop: runs 5 times per row — one iteration per column
Print one star and a space — no newline here
Inner loop completes — all 5 columns of this row are done
Move to the next line after each complete row
Outer loop completes — all 5 rows are done
Signal successful completion
End of main — output: 5 rows of 5 stars each (5x5 rectangle)

02 Right-Angle Triangle

Changing the inner loop's upper bound from a fixed number to the outer counter variable (*i*) creates a pattern that grows. Row 1 prints 1 star, row 2 prints 2 stars, and so on — because the inner loop now runs *i* times instead of a constant 5 times. This single change from '*j* <= 5' to '*j* <= *i*' is the key insight behind all variable-width patterns. Before running any pattern, trace it manually on paper: write out what *i* and *j* are for each iteration.

```
int i, j, n = 5;
for (i = 1; i <= n; i++) {
    for (j = 1; j <= i; j++) {
        printf("* ");
    }
    printf("\n");
}
```

n controls the total number of rows — change this to resize the triangle

Outer loop: row number — runs from 1 to *n*

Inner loop: runs *i* times — row 1 gets 1 star, row 2 gets 2, etc.

Print one star per column

End of inner loop

Newline after each row

Output: * / ** / *** / **** / *****

TRY THIS LIVE — Trace on Paper First

Before running the right-angle triangle code, trace it manually for *n*=4. Draw a table with columns *i*, *j*, and 'what prints'. Fill in every row: *i*=1 -> *j* goes 1 (prints *, newline). *i*=2 -> *j* goes 1,2 (prints *, *, newline). *i*=3 -> *j* goes 1,2,3 (prints *, *, *, newline). *i*=4 -> *j* goes 1,2,3,4. Then run the program and verify your trace was correct. If you can trace it, you understand it.

03 The Star Pyramid

A centred pyramid needs two pieces per row: leading spaces (to push the stars to the centre) and the stars themselves. Row *i* has (*n* - *i*) spaces and (2*i* - 1) stars. You need two separate inner loops for this: one to print spaces, another to print stars. The formula 2*i* - 1 produces the odd number sequence 1, 3, 5, 7, 9 — exactly the width each row needs to look symmetric. Always derive the formula by tracing the pattern manually before coding the inner loop condition.

PRO TIP — Derive the Formula Before You Code

For *n*=5, trace each row manually: row 1 -> 4 spaces, 1 star. row 2 -> 3 spaces, 3 stars. row 3 -> 2 spaces, 5 stars. row 4 -> 1 space, 7 stars. row 5 -> 0 spaces, 9 stars. Spot the pattern: spaces = *n*-*i*, stars = 2**i*-1.

Writing the code without this trace first is guesswork. This manual derivation habit applies to every algorithm you will ever write — not just patterns.

```
int i, j, n = 5;
for (i = 1; i <= n; i++) {
    for (j = 1; j <= n - i; j++) {
        printf(" ");
    }
    for (j = 1; j <= 2*i - 1; j++) {
        printf("* ");
    }
    printf("\n");
}
```

n = total rows; change *n* to resize the pyramid

Outer loop: one iteration per row

First inner loop: print (*n*-*i*) leading spaces for alignment

Print one space — no star

```

    }
    for (j = 1; j <= 2 * i - 1; j++) {

        printf("*");
    }
    printf("\n");
}

```

Spaces done for this row

Second inner loop: print (2i-1) stars — row 1:1, row 2:3, row 3:5, etc.

Print one star — no space between stars in a pyramid

Stars done for this row

Move to next line after each complete row

Output: centred pyramid — single star at top, widest row at base

Pattern	Outer Loop	Inner Loop Condition	Stars per Row
Rectangle (n×n)	i: 1 to n	j: 1 to n (constant)	n (constant every row)
Right-angle triangle	i: 1 to n	j: 1 to i (grows)	i (increases each row)
Inverted triangle	i: 1 to n	j: 1 to n-i+1 (shrinks)	n-i+1 (decreases each row)
Pyramid	i: 1 to n	Spaces: n-i / Stars: 2i-1	2i-1 (odd numbers: 1,3,5...)

CASE STUDY — Google Photos: Every Filter Is a Nested Loop

Context — Google Photos is used by over 500 million people in India. Every time you adjust brightness, contrast, or apply a filter in Google Photos or Snapseed on an Android device, a core algorithm runs on the image data.

Challenge — A standard 1080×1920 pixel photo contains 2,073,600 individual pixels. Applying a brightness filter means reading each pixel's red, green, and blue values and modifying them by the filter amount — for every single pixel in the image.

What they did — The image processing algorithm uses two nested loops: an outer loop from row 0 to height-1, and an inner loop from column 0 to width-1. For each pixel at position (i, j), it reads the RGB value and applies the formula. This is structurally identical to the nested loops you used to print a rectangle — the only difference is that instead of printing a star, it modifies a pixel value.

Result — A full 12-megapixel filter processes in under 100 milliseconds on a modern phone processor — achieved by the same nested loop structure you wrote today, executed at hardware speed.

Takeaway — The star grid you printed with nested loops this session and the Instagram filter on your phone are structurally the same: an outer loop for rows, an inner loop for columns, and an operation on each cell.

Source: *Google Photos Engineering Blog*, developers.google.com/; *Android Image Processing documentation*

“Every image on every screen — from a star pattern to a satellite photo — is a grid of cells processed by nested loops.”

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)**Channel** — NPTEL (official — IITs / IISc)**Watch** — Problem Solving through Programming in C — Lecture 11: Nested Loops and Patterns (IIT Kharagpur, Prof. Anupam Basu)**Link** — <https://nptel.ac.in/courses/106105085>**WHERE TO GO NEXT**

You can now make your program generate any 2D pattern by combining an outer loop for rows with one or more inner loops for columns. This exact mental model carries directly into 2D arrays.

Explore deeper — Search 'Pascal triangle in C using nested loops' — it builds the triangle you see on every maths textbook using a loop plus an array, tying this session to Session 09.

Try on your own — Print an inverted triangle (5 stars on row 1, shrinking to 1 on row 5) using the formula $j \leq n-i+1$. Then print a hollow rectangle — stars only on the border, spaces inside — for an added challenge.

Connect the dots — Next session: functions — you will learn to wrap your pattern-printing loops inside named functions, so you can call `printPyramid(5)` instead of rewriting the nested loop every time.

Going further — Look up 'Floyd's triangle in C' and 'number patterns in C' — these extend the star patterns to number sequences and reveal how the same nested loop structure can produce very different output.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a C programming teacher. Explain how nested loops work in simple Hinglish, using the analogy of a mithai shop in Ahmedabad arranged in rows and columns — the shopkeeper fills each box in a row before moving to the next row. Then describe how changing the inner loop's upper bound creates a triangular arrangement instead of a rectangle. Keep it under 150 words.

SESSION

08

60 MIN · FUNDAMENTALS

Functions

◆ Learning Objectives

- Explain why functions are needed: reusability, readability, and maintainability
- Write a complete function declaration (prototype), definition, and function call
- Pass arguments to a function and return a value using the return statement
- Distinguish between pass-by-value and pass-by-reference (via pointer) behaviour in C
- Decompose a program into multiple named functions with a single clear responsibility each

01 Why Functions?

A function is a named, reusable block of code that performs one specific task. Without functions, every program becomes a long, unbroken sequence of statements — impossible to navigate, test, or update. With functions, you break a complex problem into small, named pieces: each piece does one thing, does it well, and can be called from anywhere in the program as many times as needed. This is the same principle behind every large software system ever built.

CONCEPT — Anatomy of a Function

Three steps to use a function in C. Declaration (prototype): tells the compiler the function's name, return type, and parameter types — placed above `main()`. Definition: the actual code block that says what the function does — placed after `main()` or in a separate file. Call: the line inside `main()` (or another function) that actually runs it. You can call a function multiple times from multiple places — write once, use anywhere.

02 Declaring, Defining, and Calling

The standard C pattern is: declare the prototype above `main()`, write `main()` with the function calls, and place the full definition after `main()`. The prototype is needed because C reads the file top-to-bottom — if you call a function before its definition, the compiler does not yet know what it is. The prototype is the advance notice that says 'I promise this function exists and here is its signature.'

```
#include <stdio.h>
```

Load standard I/O library

```
int add(int a, int b);
```

DECLARATION (prototype): return type, name, parameter types — ends with semicolon

```
int main() {
```

`main()` comes before the definition — prototype makes this valid

```
int x = 5, y = 3;
int result = add(x, y);
printf("Sum: %d\n", result);
printf("Again: %d\n", add(10, 20));
return 0;
}
```

Values to pass into the function

CALL: invokes add(), passes x and y, stores the returned value

Prints: Sum: 8

Call again with different values — no new code needed

Signal successful completion

End of main

```
int add(int a, int b) {

    return a + b;
}
```

DEFINITION: full function body — a and b are local parameter variables

Computes the result and sends it back to the caller

End of add function definition

CONCEPT — void Functions vs Return Functions

int add(int a, int b) returns a value — the caller receives a result it can use. void printResult(int n) returns nothing — it performs an action (like printing) with no value sent back. Use int (or float, char, etc.) when the caller needs a result to work with. Use void when the function's job is to produce a side effect — printing, writing a file, or modifying a pointer argument.

03 Parameters, Arguments, and Return Values

Parameters are the variable names listed in the function definition — they are the function's inputs. Arguments are the actual values passed in when the function is called. Inside the function, parameters behave like regular local variables. The return statement sends a single value back to the caller and immediately exits the function — any code after return in the same function body is unreachable.

```
float calculateArea(float length, float width) {
```

Parameters: length and width (float) — inputs the function needs

```
    float area = length * width;
    return area;
}
```

Compute the result using the parameters

Send the result back to whoever called this function

Function ends — area variable is destroyed (local scope)

```
float room = calculateArea(10.5, 4.2);
```

Arguments: 10.5 and 4.2 are passed in. Return value stored in room

```
printf("Area: %.2f sq m\n", room);
```

Prints: Area: 44.10 sq m

```
printf("Area: %.2f sq m\n", calculateArea(6.0, 3.5));
```

Call directly inside printf — no intermediate variable needed

04 Pass-by-Value vs Pass-by-Reference

By default, C passes arguments by value — the function receives a copy of the variable, not the original. Modifying the parameter inside the function has no effect on the original variable in `main()`. This is safe but limiting: sometimes you genuinely need a function to modify the caller's variable — a classic example being swapping two values. To allow this, C passes the variable's memory address using the `&` operator, and the function receives a pointer (`int *`) that directly accesses the original memory location. Pointers are covered fully in Session 11; for now, understand the concept through the swap example.

Pass-by-Value (default in C)

The function receives a COPY. Changes inside the function do not affect the original. Safe — the caller's data is protected. `int x=10; swap(x,y);` — `x` is still 10 after the call. Use for most functions that just compute and return a result.

Pass-by-Reference (via pointer)

The function receives the MEMORY ADDRESS. Changes inside the function modify the original directly. `swap(&x, &y);` — `x` and `y` are actually swapped in the caller. Use when a function must modify the caller's variables (swap, sort, update).

```
// PASS-BY-VALUE — does NOT swap the originals
```

```
--- Version 1: fails to swap ---
```

```
void swapFail(int a, int b) {
```

```
    int temp = a; a = b; b = temp;
```

```
}
```

`a` and `b` are local copies — changes here do not reach `main()`

Swaps the copies — `main()`'s `x` and `y` remain unchanged

Copies are destroyed when function ends

```
// PASS-BY-REFERENCE — correctly swaps the originals
```

```
--- Version 2: works ---
```

```
void swap(int *a, int *b) {
```

```
    int temp = *a;
```

```
    *a = *b;
```

```
    *b = temp;
```

```
}
```

`*a` and `*b` are POINTERS to the original variables in `main()`

`*a` dereferences the pointer — reads the actual value of `x`

Writes `b`'s value into `x`'s memory location directly

Writes `temp` into `y`'s memory location directly

Original `x` and `y` in `main()` are now swapped

```
int x = 10, y = 20;
```

```
swap(&x, &y);
```

In `main()`: declare and initialise

Pass the ADDRESS of `x` and `y` using `&` (address-of operator)

```
printf("%d %d\n", x, y);
```

Prints: 20 10 — swap worked on the originals

TRY THIS LIVE

Write a complete program with three functions: `int square(int n)` that returns `n*n`, `float average(float a, float b, float c)` that returns `(a+b+c)/3.0`, and `void printLine()` that prints a row of 20 dashes. Call all three from `main()`

and verify the output. This single exercise covers return values, float parameters, void functions, and multiple calls — everything in this session.

CASE STUDY — Infibeam Avenues: Function Decomposition in a Live Payment Gateway

Context — Infibeam Avenues Ltd. is headquartered in Gandhinagar (GIFT City), Gujarat, and is India's first publicly listed payment gateway company (BSE: INFIBEAM). It processes digital payments for thousands of merchants across India, including government portals and large e-commerce platforms.

Challenge — A single payment transaction involves eight distinct steps: verify merchant credentials, validate card details, run fraud risk checks, calculate transaction fees, debit the amount, update the ledger, generate an invoice, and send a confirmation. Writing all this logic in one function creates a 500-line block that no single developer can safely read, test, or modify.

What they did — Infibeam's payment engine is decomposed into separate named functions — `verifyMerchant()`, `validateCard()`, `checkFraudRisk()`, `calculateFee()`, `processDebit()`, `updateLedger()`, `generateInvoice()`, `sendConfirmation()` — each independently written, tested, and deployable.

Result — When RBI updated transaction fee regulations in 2023, the engineering team modified only the `calculateFee()` function and redeployed it — without touching or retesting any other part of the payment flow. One focused change, shipped in hours instead of days.

Takeaway — Function decomposition is not just good style — it is what lets a live payment company update critical financial logic without putting the entire Rs 1,000-crore transaction flow at risk.

Source: Infibeam Avenues Annual Report 2022-23, infibeam.com; RBI Payment Aggregator Guidelines 2022

“A function is a promise: I will do this one thing reliably, every single time you call me.”

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Problem Solving through Programming in C — Lecture 14: Functions — Declaration, Definition, Call (IIT Kharagpur, Prof. Anupam Basu)

Link — <https://nptel.ac.in/courses/106105085>

WHERE TO GO NEXT

You can now break any program into named, reusable blocks. From here, every session builds on functions — arrays and strings will be passed into them, pointers will unlock pass-by-reference fully, and structures will let you create powerful custom data types.

Explore deeper — Search 'recursion in C simple example' — a function that calls itself. It is one of the most powerful ideas in programming and builds directly on everything you learned today.

Try on your own — Refactor the grade classifier from Session 05 into two functions: `char getGrade(int marks)` that returns the grade letter, and `void printResult(int marks)` that calls `getGrade()` and prints a formatted result line. `main()` should only call `printResult()`.

Connect the dots — Next session: arrays — you will pass arrays into functions, and the distinction between pass-by-value and pass-by-reference will become even more important because arrays always pass their address.

Going further — Look up 'function pointers in C' — storing the address of a function in a variable, then calling it through that variable. This is how callbacks, event handlers, and plugin architectures work.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a C programming teacher. Explain the difference between pass-by-value and pass-by-reference in simple Hinglish. Use the analogy of giving someone a photocopy of your Aadhaar card (they can write on it but your original is unchanged) vs giving them your original card directly (they can actually modify it). Keep it under 150 words.

SESSION

09

60 MIN · FUNDAMENTALS

Arrays

◆ Learning Objectives

- Explain what an array is and why it is needed over declaring many separate variables
- Declare and initialise a 1D array with a fixed size and initial values
- Access and modify array elements using zero-based index notation
- Traverse an array using a for loop to read, accumulate, and print values
- Declare and access a 2D array to represent tabular data like student marks

01 What is an Array?

Suppose you need to store screen time for 7 days. Without arrays, you would declare seven separate variables: day1, day2, day3 ... day7. With an array, you declare one variable that holds all seven values in a single block of contiguous memory. An array is a collection of elements of the same data type, stored sequentially, and accessed by a numerical index. The index always starts at 0 — not 1.

CONCEPT — Array Declaration and Zero-Based Indexing

`int screenTime[7];` declares an integer array named `screenTime` with 7 slots — indices 0 through 6. `screenTime[0]` is the first element, `screenTime[6]` is the last. There is NO `screenTime[7]` — accessing it reads or writes unknown memory and causes undefined behaviour. This is one of the most common C bugs: always loop from `i=0` to `i<size`, never `i<=size`.

ARRAY BUGS IN THE TOP 25

Top 5

Out-of-bounds array access (CWE-125, CWE-787) is consistently ranked in the Top 5 most critical software weaknesses — MITRE CWE Top 25, 2023

02 Declaring, Initialising, and Traversing a 1D Array

An array can be declared with or without initial values. With initialisation: `int screenTime[7] = {3, 5, 2, 4, 6, 7, 3};` — all seven values are set at declaration. Without initialisation: `int screenTime[7];` — slots hold garbage values until you assign them. If you provide fewer initialisers than the size, C fills the remaining slots with zero. Traversal always uses a for loop counting from 0 to `size-1`, making the loop condition `i < 7` rather than `i <= 7`.

```
#include <stdio.h>
int main() {
int screenTime[7] = {3, 5, 2, 4, 6, 7, 3};
    int i, total = 0;
    float average;
printf("--- 7-Day Screen Time Report ---\n");
    for (i = 0; i < 7; i++) {
printf("Day %d: %d hours\n", i + 1, screenTime[i]);
        total += screenTime[i];
    }
    average = (float)total / 7;

    printf("Total   : %d hours\n", total);
printf("Average: %.1f hours/day\n", average);
    return 0;
}
```

Load standard I/O library

Program entry point

7-element array: indices 0 to 6

Loop counter and total accumulator

Float for the average — int would drop the decimal

Report header

$i < 7$, NOT $i \leq 7$ — valid indices are 0 to 6 only

$i+1$ for display (Day 1-7); `screenTime[i]` for the stored value

Accumulate — shorthand for `total = total + screenTime[i]`

All 7 days processed

(float) cast forces decimal division — without it, integer division drops the decimal

Prints 30

Prints 4.3

Signal successful completion

End of main

WARNING — The Off-By-One Error

`int arr[5] = {10, 20, 30, 40, 50};` — valid indices are 0, 1, 2, 3, 4. Writing `arr[5]` accesses memory outside the array — C will not warn you at compile time, but the program may crash, print garbage, or corrupt other variables silently. This is called a buffer overflow and is one of the most exploited security vulnerabilities in real software. The fix is always: loop while $i < \text{size}$, never $i \leq \text{size}$.

03 2D Arrays

A 2D array extends the concept to rows and columns — it is an array of arrays. Declare it as `int marks[3][3]` — 3 rows, 3 columns. Access it as `marks[row][column]`. You traverse it with nested loops: the outer loop for rows, the inner loop for columns — exactly the nested loop pattern from Session 07, now applied to real data instead of stars.

CONCEPT — 2D Array Declaration and Access

`int marks[3][3];` — 3 rows, 3 columns, 9 elements total. `marks[0][0]` is row 0 column 0 (first element). `marks[2][2]` is row 2 column 2 (last element). Initialise as: `int marks[3][3] = {{85,76,92},{70,88,65},{95,82,78}};` — each inner set of braces is one row. Memory is still contiguous — C stores it row by row.

```
int marks[3][3] = {
    {85, 76, 92},
    {70, 88, 65},
    {95, 82, 78}
};
int i, j;
for (i = 0; i < 3; i++) {
    printf("Student %d: ", i + 1);
    for (j = 0; j < 3; j++) {
        printf("%3d ", marks[i][j]);
    }
    printf("\n");
}
```

2D array: 3 students (rows) x 3 subjects (columns)
 Student 1: Maths 85, Science 76, English 92
 Student 2
 Student 3 — no comma after the last row
 End of 2D initialisation
 Loop counters — i for rows (students), j for columns (subjects)
 Outer loop: one pass per student
 Print student number
 Inner loop: one pass per subject
 %3d right-aligns in 3 characters — cleaner column output
 Inner loop ends — all subjects for this student printed
 Move to next line after each student row
 Outer loop ends — all 3 students printed

TRY THIS LIVE

Extend the 2D marks program: add a third loop that calculates each student's total marks and average, then prints a summary row after each student's marks. You will need a sum variable that resets to 0 at the start of each outer iteration. Then find the student with the highest average — that requires tracking a max variable across the outer loop.

CASE STUDY — GSEB: Processing Exam Results for 13 Lakh Students

Context — The Gujarat Secondary and Higher Secondary Education Board (GSEB), Gandhinagar, conducts Class 10 (SSC) and Class 12 (HSC) board exams for over 13 lakh students across all 33 districts of Gujarat every year.

Challenge — After the exam, marks from every centre must be loaded, validated, aggregated by school and district, and used to generate individual results — all before the public announcement date. Processing must be accurate for 13 lakh individual records simultaneously.

What they did — The result processing system uses 2D data structures: a marks[student_id][subject] grid to store raw scores, traversed by nested loops to calculate totals, classify pass/fail, and assign grades. District-level and school-level summary arrays are computed in a second pass over the same data.

Result — GSEB generates and publishes individual results for 13+ lakh students within hours of the result announcement — driven by the same array traversal and accumulation logic you practised today.

Takeaway — The screenTime[7] array you built today and the marks[1300000][6] grid at GSEB are the same data structure — one just has more rows.

Source: GSEB Annual Report, gseb.org; Gujarat Education Department statistics, 2023

“An array is the bridge between a single value and a million — same logic, different scale.”

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Problem Solving through Programming in C — Lecture 16: Arrays — 1D and 2D (IIT Kharagpur, Prof. Anupam Basu)

Link — <https://nptel.ac.in/courses/106105085>

WHERE TO GO NEXT

You can now store, traverse, and aggregate collections of data using 1D and 2D arrays. This is the foundation for the mini project at the end of this module.

Explore deeper — Search 'sizeof operator in C for arrays' — it lets you compute the number of elements in an array without hardcoding the size, which prevents the off-by-one errors you need to avoid.

Try on your own — Write a program that reads 7 daily temperatures from the user using scanf inside a for loop, stores them in a float array, then prints the maximum, minimum, and average temperatures.

Connect the dots — Next session: strings — a string in C is simply a char array with a special null character at the end. Everything you learned about array traversal applies directly.

Going further — Research 'dynamic memory allocation in C: malloc and free' — arrays have a fixed size at compile time, but malloc lets you allocate an array whose size is only known at runtime.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a C programming teacher. Explain 1D and 2D arrays in simple Hinglish. Use the example of a weekly timetable: a 1D array is one class's daily schedule, a 2D array is the full school timetable with rows for classes and columns for periods. Also explain why indexing starts at 0 and not 1. Keep it under 150 words.

SESSION

10

60 MIN · FUNDAMENTALS

Strings

◆ Learning Objectives

- Explain how C represents a string as a null-terminated char array
- Declare and initialise a string using a char array
- Read a full-name string including spaces using fgets()
- Apply strlen(), strcpy(), strcmp(), and strcat() from the string.h library
- Build a program that constructs a username from the user's first name and department code

01 Strings in C — char Arrays with a Terminator

C has no built-in string type. A string is a char array where the last element is always the null character '\0' (ASCII value 0). This null terminator is how C knows where the string ends — without it, functions like printf would keep reading bytes past the array boundary. When you write `char name[6] = "Rahul";`, C stores five characters ('R','a','h','u','l') plus '\0' at index 5. Always declare your char array at least one element larger than the string you plan to store.

CONCEPT — Null Terminator and String Declaration

`char name[50] = "Rahul";` — the compiler automatically adds \0 at position 5. `char name[50];` followed by `name[0]='R'; ...` does NOT add \0 unless you do it explicitly. `char name[] = "Rahul";` — the compiler auto-sizes the array to 6 (5 chars + 1 for \0). Never write `char name[5] = "Rahul";` — there is no room for \0 and the string is invalid.

```
#include <stdio.h>
int main() {
    char name[50] = "Rahul Mehta";
    char city[] = "Ahmedabad";
    printf("Name: %s\n", name);
    printf("City: %s\n", city);
    printf("First char: %c\n", name[0]);
    printf("Last char : %c\n", name[10]);
    return 0;
}
```

Load standard I/O library

Program entry point

String literal initialisation — \0 added automatically at end

Auto-sized array: 10 elements (9 chars + \0)

%s prints chars from the start address until it hits \0

Prints: City: Ahmedabad

Access individual char — prints: R

Index 10 = 'a' (last char before \0 in 'Rahul Mehta')

Signal successful completion

End of main

02 Reading Strings from the User

You already know `scanf("%d", &n)` for integers. For strings, `scanf("%s", name)` reads characters until it hits a space or newline — which means it only reads one word. To read a full name with spaces (like 'Rahul Mehta'), use `fgets(name, 50, stdin)` instead. The three arguments are: the array to write into, the maximum number of characters to read (including `\0`), and `stdin` (the keyboard). `fgets` is safer and handles multi-word input.

```
char firstName[30], fullName[60];
printf("Enter first name (one word): ");
scanf("%s", firstName);

printf("Enter full name (with spaces): ");
scanf(" ");

fgets(fullName, 60, stdin);

fullName[strcspn(fullName, "\n")] = '\0';

printf("First: %s\n", firstName);
printf("Full : %s\n", fullName);
```

Two char arrays for different read methods

Single-word prompt

No & needed — array name is already an address. Stops at space.

Multi-word prompt

Consume the leftover newline from previous `scanf` before `fgets`

Reads up to 59 chars including spaces — stops at newline or end of buffer

`fgets` keeps the `\n` — this line strips it. Requires `#include <string.h>`

Prints first word only

Prints entire name including spaces

WARNING — `scanf` Stops at Spaces

`scanf("%s", name)` with input 'Rahul Mehta' stores only 'Rahul' — the space terminates the read. 'Mehta' is left in the input buffer and will be consumed by the next `scanf` call, causing confusing bugs. For full names and any input that may contain spaces, always use `fgets()`. Also: do not pass `&name` to `scanf` for strings — `name` is already the address of the array. Writing `scanf("%s", &name)` is technically wrong even if some compilers accept it.

03 Common String Functions — `string.h`

The `string.h` header provides a set of standard functions for working with strings. You never need to write character-by-character loops for common operations — these functions are optimised and reliable. Always add `#include <string.h>` at the top of any file that uses string functions. The four most used functions are `strlen`, `strcpy`, `strcmp`, and `strcat` — master these and you can handle most string tasks in C.

Function	Purpose	Example	Result
<code>strlen(s)</code>	Length of string (excluding <code>\0</code>)	<code>strlen("Rahul")</code>	5
<code>strcpy(dst, src)</code>	Copy <code>src</code> into <code>dst</code>	<code>strcpy(dest, "TOPS")</code>	<code>dest = "TOPS"</code>

Function	Purpose	Example	Result
strcmp(s1, s2)	Compare: 0 if equal, <0 or >0 if not	strcmp("abc","abc")	0 (equal)
strcat(dst, src)	Append src to end of dst	strcat("Hello"," World")	"Hello World"

```
#include <stdio.h>
#include <string.h>
int main() {
    char s1[50] = "Ahmedabad";
    char s2[50];
    printf("Length: %lu\n", strlen(s1));
    strcpy(s2, s1);
    strcat(s2, " - Gujarat");
    printf("%s\n", s2);
    if (strcmp(s1, "Ahmedabad") == 0)
        printf("Match!\n");
    return 0;
}
```

Standard I/O
Required for all string functions
Program entry point
Source string
Destination — uninitialised
Prints 9 — does not count \0
Copy s1 into s2 — s2 now contains 'Ahmedabad'
Append to s2 — s2 becomes 'Ahmedabad - Gujarat'
Prints: Ahmedabad - Gujarat
strcmp returns 0 when strings are identical
Prints: Match!
Signal successful completion
End of main

04 Worked Example — Username Generator

A username system is a practical demonstration of all four string operations in sequence: read the user's name with fgets, measure its length with strlen, convert to lowercase with a loop and tolower(), trim it with strcpy if too long, and append the department code and a number with strcat. The result is a system similar to what TOPS's own student portal uses to generate login IDs.

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
int main() {
    char firstName[30], dept[10], username[50];
    Three char arrays
    printf("Enter first name: ");
    scanf("%s", firstName);
    printf("Enter dept code (e.g. CS, IT, DS): ");
    Department code prompt
    scanf("%s", dept);
    int i;
    for (i = 0; firstName[i] != '\0'; i++)
        firstName[i] = tolower(firstName[i]);
```

Standard I/O
String functions
For tolower() — converts chars to lowercase
Program entry point
Prompt
Single word — no spaces in first name
Read department code
Loop counter for lowercase conversion
Loop until null terminator — converts entire string
Convert each character to lowercase in-place

```
strcpy(username, firstName);  
strcat(username, ".");  
strcat(username, dept);  
strcat(username, "@tops");  
printf("Username: %s\n", username);  
return 0;  
}
```

Start username with lowercase first name
Append separator dot
Append department code
Append domain
Prints e.g.: rahul.CS@tops
Signal successful completion
End of main

TRY THIS LIVE

Extend the username generator: use `strlen()` to check if `firstName` is longer than 8 characters, and if so, truncate it to 8 before building the username — `firstName[8] = '\0'`; does the job. Then add `strcmp()` to reject the username if it matches a reserved word like 'admin' or 'root', and prompt the user to enter a different name. These two additions cover the real-world username validation logic used in most login systems.

CASE STUDY — NPCI UPI: String Operations Behind 30 Crore UPI IDs

Context — The National Payments Corporation of India (NPCI) built the Unified Payments Interface (UPI) system. Every user of PhonePe, GPay, Paytm, or BHIM gets a UPI ID in the format `name@bankhandle` — for example, `rahul.mehta@okicici` or `9876543210@ybl`.

Challenge — Creating a unique, human-readable UPI ID for each new user requires taking their name or mobile number, normalising it (removing spaces, converting to lowercase), validating it (checking length, allowed characters), and appending the correct bank handle — all through string operations.

What they did — NPCI's ID generation system uses exactly the operations you learned today: `strlen()` for length validation, `strcpy()` and `strcat()` to assemble the ID, `tolower()` for normalisation, and `strcmp()` to check for duplicates against existing registered IDs in the database.

Result — Over 30 crore unique UPI IDs have been generated using these string operations, enabling 1,000 crore+ UPI transactions per month as of 2024 — making India the world's largest real-time payments market.

Takeaway — Every UPI ID in India — including the one on your own PhonePe account — was assembled using `strlen`, `strcpy`, `strcat`, and `strcmp`. The functions you practised today.

Source: NPCI UPI Transaction Data, npci.org.in; RBI Payment System Indicators, 2024

“A string in C is just an array of characters — but an array of characters is the foundation of every name, message, and ID in every system ever built.”

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Problem Solving through Programming in C — Lecture 18: Strings and String Functions (IIT Kharagpur, Prof. Anupam Basu)

Link — <https://nptel.ac.in/courses/106105085>

WHERE TO GO NEXT

You can now store, read, manipulate, and compare text data in C. Strings and arrays together cover the vast majority of real-world data types.

Explore deeper — Search 'strtok function in C' — it splits a string by a delimiter (like splitting 'rahul.mehta@tops' by '.') and is essential for parsing structured text like CSV files.

Try on your own — Write a program that reads a full sentence from the user with fgets, counts the number of words (hint: count spaces + 1), and also counts the total characters excluding the null terminator.

Connect the dots — Next session: pointers — you will learn that the array name (like 'name' in char name[50]) is actually a pointer to the first element. This explains why you did not use & with scanf for strings.

Going further — Look up 'sprintf in C' — it works like printf but writes into a char array instead of the screen. This is how C programs build formatted strings dynamically, exactly like string interpolation in Python.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a C programming teacher. Explain null-terminated strings in C in simple Hinglish. Use the analogy of a message written on paper where the last character is a full stop that tells the reader 'this is where the text ends.' Then explain why strlen counts characters but not the full stop (null terminator). Keep it under 150 words.

SESSION

11

60 MIN · FUNDAMENTALS

Pointers

◆ Learning Objectives

- Explain what a pointer is: a variable that stores a memory address, not a value
- Use the address-of operator (&) to get a variable's address and the dereference operator (*) to access the value at that address
- Declare a pointer variable with the correct data type
- Perform basic pointer arithmetic to traverse an array
- Implement the swap function using pointers — completing the pass-by-reference concept from Session 08

01 What is a Pointer?

Every variable you declare is stored at a specific location in RAM. That location has a numerical address — like a flat number in a housing complex. A pointer is a variable whose value is one of those addresses. Instead of storing 42 or 'A', a pointer stores the address of another variable. This is the mechanism behind pass-by-reference, arrays, dynamic memory, and virtually every advanced C concept — understanding pointers is the turning point in learning C.

CONCEPT — The Two Pointer Operators: & and *

& (address-of): placed before a variable name, it returns the memory address of that variable. `int x = 10; printf("%p", &x);` prints something like `0x7ffde1a4`. * (dereference): placed before a pointer variable, it accesses the value stored at the address the pointer holds. If `p` stores the address of `x`, then `*p` gives you 10 — the actual value of `x`. Reading the address is `&`; reading through the address is `*`.

POINTER BUG SHARE

~70%

Pointer-related errors account for approximately 70% of critical security vulnerabilities in C and C++ codebases — Microsoft Security Response Centre, 2019

02 Declaring and Using Pointers

A pointer is declared by placing an asterisk (*) between the data type and the pointer name: `int *p`; — read this as 'p is a pointer to an int.' The data type tells the compiler how many bytes to read when you dereference the pointer. An int pointer reads 4 bytes; a char pointer reads 1 byte. Always initialise a pointer before dereferencing it — an uninitialised pointer holds a random address and dereferencing it causes a crash (segmentation fault).

<code>#include <stdio.h></code>	Standard I/O library
<code>int main() {</code>	Program entry point
<code> int x = 10;</code>	Declare an integer variable
<code> int *p;</code>	Declare a pointer to int — p can store the address of an int
<code> p = &x;</code>	Assign the address of x to p — p now 'points to' x
<code> printf("Value of x : %d\n", x);</code>	Prints 10 — direct access
<code> printf("Address of x: %p\n", &x);</code>	Prints a hex address like 0x7ffd... (will vary)
<code> printf("p holds : %p\n", p);</code>	Same address as &x — p points to x
<code> printf("*p (via ptr): %d\n", *p);</code>	Dereference p — reads the value AT that address: 10
<code> *p = 25;</code>	Write through pointer — modifies x directly
<code> printf("x is now : %d\n", x);</code>	Prints 25 — x was changed via the pointer
<code> return 0;</code>	Signal successful completion
<code>}</code>	End of main

CONCEPT — Reading Pointer Declarations

Read from right to left: `int *p` — 'p is a (*) pointer to (int) an integer.' `float *f` — 'f is a pointer to a float.' `char *c` — 'c is a pointer to a char.' The * in a declaration means 'pointer to' and is part of the type — it is NOT the dereference operator here. The dereference * only works after the pointer is already declared and initialised.

03 Pointer Arithmetic

When you add 1 to a pointer (`ptr++`), C does not add 1 byte — it adds the size of the pointed-to type. For an int pointer, `ptr++` moves 4 bytes forward. For a char pointer, it moves 1 byte. This is how array traversal works at the hardware level: array elements are stored contiguously in memory, and incrementing a pointer steps from one element to the next. The array name itself is a constant pointer to the first element.

<code>int arr[5] = {10, 20, 30, 40, 50};</code>	5-element array stored contiguously in memory
<code>int *ptr = arr;</code>	arr is the address of arr[0] — ptr points to first element
<code>printf("%d\n", *ptr);</code>	Prints 10 — arr[0]
<code>printf("%d\n", *(ptr + 1));</code>	Prints 20 — arr[1]: +1 moves 4 bytes (sizeof int) forward
<code>printf("%d\n", *(ptr + 2));</code>	Prints 30 — arr[2]
<code>ptr++;</code>	Advance ptr to next element — now points to arr[1]
<code>printf("%d\n", *ptr);</code>	Prints 20
<code>printf("%d\n", arr[3]);</code>	arr[3] and *(arr+3) are identical — bracket notation is syntactic sugar

04 Completing the Swap — Pass-by-Reference Fully Explained

In Session 08 you learned that `swapFail(x, y)` does not work because C copies the values. Now you have the tools to understand why `swap(&x, &y)` works: you pass the addresses of x and y to the function. Inside the function, *a and *b

are pointers to the original variables — modifying `*a` directly modifies `x` in `main()`. This is the complete picture of pass-by-reference in C: you pass the address, and the function reaches through that address to modify the original.

```
#include <stdio.h>
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}
int main() {
    int x = 10, y = 20;
    printf("Before: x=%d y=%d\n", x, y);
    swap(&x, &y);

    printf("After : x=%d y=%d\n", x, y);
    return 0;
}
```

Standard I/O library
Accepts pointers — `a` holds address of `x`, `b` holds address of `y`
Read the value at address `a` (i.e., `x`) into `temp`
Write the value at `b` (i.e., `y`) into the location `a` points to (`x`)
Write `temp` into the location `b` points to (`y`)
`x` and `y` in `main()` are now actually swapped
Program entry point
Two integers to swap
Prints: Before: `x=10 y=20`
Pass addresses — `&x` is the address of `x`, `&y` is the address of `y`
Prints: After : `x=20 y=10` — swap worked on originals
Signal successful completion
End of `main`

TRY THIS LIVE

Write a function `void getMinMax(int arr[], int n, int *min, int *max)` that finds the minimum and maximum of an integer array and returns both values through pointer parameters. In `main()`, declare `int smallest, largest`; and call `getMinMax(arr, 7, &smallest, &largest)`; then print both. This is the standard C pattern for returning multiple values from a function — you will use it frequently in real programs.

CASE STUDY — Embedded C in Gujarat's Automotive Sector

Context — Gujarat is a major automotive manufacturing hub — Tata Motors (Sanand), Suzuki (Hansalpur/Becharaji), and Ford (previously Sanand) have all operated assembly plants in the state. Every modern vehicle has an Engine Control Unit (ECU) — a microcontroller running embedded C firmware.

Challenge — An ECU must read live sensor data (fuel pressure, crankshaft position, throttle position) from hardware registers at specific memory addresses and compute injection timing in microseconds. Waiting for data to be copied by value is too slow — direct memory access is required.

What they did — ECU firmware accesses hardware registers using memory-mapped pointers: `volatile int *throttleSensor = (int *)0x40021008`; reads the sensor value directly from its fixed hardware address. The `volatile` keyword tells the compiler not to cache the value — it may change any time hardware updates it. This is pointer usage in its most literal form.

Result — Real-time engine control at microsecond precision — impossible without direct pointer-based memory access. Every car manufactured in Gujarat's plants contains microcontrollers running this kind of pointer-driven embedded C code.

Takeaway — The `*ptr = 25` you wrote today and the `*throttleSensor` read in an ECU firmware file are the same pointer dereference — one on a PC, one controlling a car engine.

Source: Embedded C Programming for Automotive Applications, SAE International; Gujarat Investment Summit

automotive data, 2023

“A pointer does not hold a value — it holds the address of a value. That single distinction unlocks half of what C can do.”

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Problem Solving through Programming in C — Lecture 20: Pointers — Declaration, Dereferencing, Arithmetic (IIT Kharagpur, Prof. Anupam Basu)

Link — <https://nptel.ac.in/courses/106105085>

WHERE TO GO NEXT

Pointers are the mechanism behind arrays, strings, pass-by-reference, dynamic memory, and linked lists. You now have the foundation to understand every advanced C concept.

Explore deeper — Search 'NULL pointer in C' — what it is, when to use it, and why dereferencing NULL causes a segmentation fault. Every pointer that might not point to valid memory should be checked for NULL first.

Try on your own — Write a function `void reverseArray(int *arr, int n)` that reverses an integer array in-place using pointer notation — increment a pointer from the front and decrement one from the back, swapping values at each step.

Connect the dots — Next session: structures — you can now understand why struct pointers (struct Student *s) and the arrow operator (s->marks) work the way they do.

Going further — Look up 'double pointers in C (int **)' — a pointer to a pointer. This is how 2D arrays are handled in dynamic memory allocation and how command-line arguments (`char *argv[]`) are structured.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a C programming teacher. Explain pointers in simple Hinglish using the analogy of a house and its address: the variable is the house, its value is who lives inside, and the pointer is a piece of paper with the house's address written on it. Explain & as writing down the address and * as going to that address to see who lives there. Keep it under 150 words.

SESSION

12

60 MIN · FUNDAMENTALS

Structures

◆ Learning Objectives

- Explain why structures are needed: grouping related data of different types into one unit
- Declare a struct, define variables of that type, and access members using the dot operator
- Initialise a struct at declaration and assign members individually
- Create an array of structs to store multiple records and traverse it with a loop
- Declare and use a nested struct where one struct is a member of another

01 Why Structures?

Arrays group multiple values of the same type. But a student record needs different types together: a name (char array), a roll number (int), marks (float), and a grade (char). You could declare four separate arrays and keep them in sync manually — but that approach breaks the moment you need to sort or pass the data. A structure groups all of these fields into a single named unit, letting you treat a student as one cohesive object rather than four disconnected variables.

Without struct — disconnected

`char names[30][50]; int rollNos[30]; float marks[30]; char grades[30];` — four arrays that must always be kept in sync. Moving student 5's data means updating all four arrays. Error-prone and unreadable.

With struct — one unit

`struct Student class[30];` — one array of 30 complete student records. Each element has name, rollNo, marks, and grade bundled together. Moving one student means moving one struct element.

CONCEPT — struct Syntax and the Dot Operator

`struct TagName { member1; member2; };` declares the template — no memory is allocated yet. `struct TagName variableName;` creates a variable of that type — this is when memory is allocated. `variableName.member` accesses a specific field using the dot operator (.). A struct can hold any mix of types: int, float, char arrays, other structs, and even pointers.

02 Declaring and Using a struct

Define the struct template outside `main()` — this makes it available throughout the file. Then declare variables of that struct type inside `main()`. You can initialise a struct at declaration using braces in the same order as the member declarations, or assign each member individually using the dot operator. Both approaches are valid; individual assignment is more readable when there are many fields.

```
#include <stdio.h>
#include <string.h>
struct Student {
    char name[50];
    int rollNo;
    float marks;
    char grade;
};
int main() {
    struct Student s1 = {"Rahul Mehta", 101, 87.5, 'A'};
    struct Student s2;

    strcpy(s2.name, "Priya Shah");
    s2.rollNo = 102;
    s2.marks = 72.0;
    s2.grade = 'B';
    printf("%s | %d | %.1f | %c\n", s1.name, s1.rollNo, s1.marks, s1.grade);
    printf("%s | %d | %.1f | %c\n", s2.name, s2.rollNo, s2.marks, s2.grade);
    return 0;
}
```

Standard I/O library
For strcpy — needed to assign string members
Define the struct template — no memory allocated here
Member: student's name (char array)
Member: roll number
Member: marks out of 100
Member: single letter grade
End of struct definition — note the semicolon
Program entry point
Initialise at declaration — order must match member order
Declare without initialising — members hold garbage values
Assign string member — cannot use = for char arrays
Assign int member with dot operator
Assign float member
Assign char member
Print s1: Rahul Mehta | 101 | 87.5 | A
Print s2: Priya Shah | 102 | 72.0 | B
Signal successful completion
End of main

03 Array of Structures

To store multiple records, declare an array of structs: `struct Student class[30]`; gives you 30 complete student records. Access the *i*-th student's marks as `class[i].marks` — first the array index, then the dot operator, then the member name. Traversal with a for loop is identical to any other array, but each element now carries a full record of mixed types.

```
struct Student class[3] = {
    {"Rahul Mehta", 101, 87.5, 'A'},
    {"Priya Shah", 102, 72.0, 'B'},
    {"Arjun Patel", 103, 55.5, 'C'}
};
int i;
printf("%-20s %6s %7s %6s\n", "Name", "Roll", "Marks", "Grade");
for (i = 0; i < 3; i++) {
    printf("%-20s %6d %7.1f %6c\n",
        class[i].name, class[i].rollNo, class[i].marks, class[i].grade);
}
```

Array of 3 structs — initialise all at once
Student 0
Student 1
Student 2 — no comma after last element
End of array initialisation
Loop counter
Formatted header — %-20s left-aligns in 20 chars
Traverse all 3 student records
Print one record per line
Access each member: array index then dot then member name

```
}
```

All records printed

04 Nested Structures

A struct member can itself be another struct — this is called nesting. For example, a Student struct can contain an Address struct to store the student's city, state, and pin code. This keeps related sub-data grouped together within the parent record. Access nested members by chaining dot operators: `student.addr.city` reads the city field inside the `addr` member of the student struct.

<code>struct Address {</code>	Define the inner struct first
<code>char city[30];</code>	City name
<code>char state[30];</code>	State name
<code>int pincode;</code>	6-digit pin code
<code>};</code>	End of Address struct
<code>struct Student {</code>	Outer struct — uses Address as a member type
<code>char name[50];</code>	Student name
<code>float marks;</code>	Marks
<code>struct Address addr;</code>	Nested struct member — full Address embedded here
<code>};</code>	End of Student struct
<code>struct Student s;</code>	Declare one student variable
<code>strcpy(s.name, "Rahul Mehta");</code>	Assign outer member
<code>s.marks = 87.5;</code>	Assign outer member
<code>strcpy(s.addr.city, "Ahmedabad");</code>	Access nested member — dot.dot notation
<code>s.addr.pincode = 380001;</code>	Assign nested int member
<code>printf("%s, %s — %d\n", s.name, s.addr.city, s.addr.pincode);</code>	
Prints: Rahul Mehta, Ahmedabad — 380001	

TRY THIS LIVE

Extend the array of 3 students: add a function `float classAverage(struct Student s[], int n)` that computes the average marks across all students, and `void topStudent(struct Student s[], int n)` that prints the name and marks of the student with the highest marks. Pass the array and its size to both functions from `main()`. This exercise combines structs, arrays, and functions from Sessions 08–12 into a single integrated program.

CASE STUDY — Ahmedabad Municipal Corporation: Property Records as Structs

Context — The Ahmedabad Municipal Corporation (AMC) manages one of India's largest city administrations, including property tax assessment for over 15 lakh properties in Ahmedabad — from residential flats in Bodakdev to commercial units in GIDC estates.

Challenge — Each property record requires multiple data types together: owner name (text), property type (single char: R/C/I for residential/commercial/industrial), area in square metres (float), zone number (int), and

annual tax amount (float). Storing these as five disconnected arrays for 15 lakh records is unmanageable.

What they did — AMC's property database uses a record structure identical to a C struct: one logical unit per property containing all five fields together. Database operations — sorting by zone, filtering by type, computing tax totals — all work on the complete record as one unit, just as struct operations work on the full struct.

Result — AMC processes property tax assessments, notices, and receipts for 15+ lakh Ahmedabad properties each year — with every record structured exactly as the Student struct you wrote today, just with different field names.

Takeaway — struct Student and AMC's property record are the same concept: different named fields of different types, grouped into one logical unit that can be copied, compared, and passed as one.

Source: Ahmedabad Municipal Corporation Annual Report, ahmedabadcity.gov.in; AMC Smart City Portal data, 2023

“A struct is the C way of saying: these things belong together — keep them together.”

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Problem Solving through Programming in C — Lecture 24: Structures and Nested Structures (IIT Kharagpur, Prof. Anupam Basu)

Link — <https://nptel.ac.in/courses/106105085>

WHERE TO GO NEXT

You can now model real-world entities — students, products, employees, transactions — as structs in C. This is the conceptual foundation for classes and objects in OOP (Module 4).

Explore deeper — Search 'typedef struct in C' — typedef lets you write Student s1 instead of struct Student s1, which is cleaner and is the convention in most real C codebases.

Try on your own — Add a function void displayWithRank(struct Student s[], int n) that assigns ranks (1st, 2nd, 3rd) based on marks and prints each student with their rank. You will need to sort the array — research 'bubble sort in C' to implement the sorting step.

Connect the dots — Next session: file handling — you will save your struct array to a file and load it back, giving your student records system true data persistence.

Going further — Look up 'struct pointers and the arrow operator in C' — struct Student *s; followed by s->marks instead of (*s).marks. This is how structs are used in linked lists and dynamic data structures.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a C programming teacher. Explain structures in C in simple Hinglish. Use the analogy of a student's hall ticket (pravesha patra) which has multiple fields — name, roll number, subject, exam date — all belonging to one student. Contrast this with keeping a separate notebook for each piece of information. Explain why grouping them into one struct is better. Keep it under 150 words.

SESSION

13

60 MIN · FUNDAMENTALS

File Handling

◆ Learning Objectives

- Explain why file handling is essential: programs lose all data when they end without saving to a file
- Open a file using `fopen()` with the correct mode and store the `FILE` pointer
- Write formatted text to a file using `fprintf()` and close it with `fclose()`
- Read lines from a file using `fgets()` in a loop until end-of-file
- Build a program that writes daily goals to a `.txt` file and reads them back

01 Why Files? The Persistence Problem

Every program you have written so far loses all its data the moment it ends. Variables live in RAM — temporary memory that is wiped when the process exits. Files live on the storage drive — persistent memory that survives reboots, exits, and power cuts. Any useful real-world application — a note-taking app, a student tracker, a billing system — must save data to a file so it can be loaded again in the next session. File handling is what transforms a demo into a real program.

CONCEPT — The `FILE` Pointer and `fopen()`

`FILE *fp = fopen("filename.txt", "mode");` — `fopen()` opens a file and returns a `FILE` pointer that represents the connection. All subsequent read/write operations use this pointer. If the file cannot be opened (wrong path, no permission, disk full), `fopen()` returns `NULL`. Always check: `if (fp == NULL) { handle error }`. Never skip this check — a `NULL` file pointer crash is one of the most common runtime errors.

Mode	Action	File Exists?	File Missing?
"r"	Read only	Opens it	Returns <code>NULL</code> — error
"w"	Write (overwrite)	Clears and overwrites	Creates a new file
"a"	Append (add to end)	Adds to end, keeps content	Creates a new file
"r+"	Read and write	Opens as-is	Returns <code>NULL</code> — error
"w+"	Write and read	Clears and rewrites	Creates a new file

02 Opening, Writing, and Closing a File

The file workflow has three steps: open, operate, close. Always close the file with `fclose(fp)` when done — this flushes the internal buffer to disk and releases the file handle. A program that opens files without closing them may lose the last few writes (they stay in the buffer and never reach the disk) and eventually run out of file handles if many files are opened in a loop.

<code>#include <stdio.h></code>	Standard I/O — FILE type and all file functions are here
<code>int main() {</code>	Program entry point
<code>FILE *fp;</code>	Declare the FILE pointer
<code>fp = fopen("daily_goals.txt", "w");</code>	Open for writing — creates file if missing, clears if exists
<code>if (fp == NULL) {</code>	Always check — never skip this
<code>printf("Error: Could not open file.\n");</code>	
<code>Inform the user</code>	
<code>return 1;</code>	Return non-zero to signal error to the OS
<code>}</code>	File is open and fp is valid from here
<code>fprintf(fp, "Goal 1: Complete the C arrays session\n");</code>	
<code>fprintf works like printf but writes to the file, not screen</code>	
<code>fprintf(fp, "Goal 2: Practice pointer exercises\n");</code>	
<code>Write second goal</code>	
<code>fprintf(fp, "Goal 3: Review today's session notes\n");</code>	
<code>Write third goal</code>	
<code>fclose(fp);</code>	Flush buffer to disk and release the file handle — mandatory
<code>printf("Goals saved to daily_goals.txt\n");</code>	
<code>Confirm success to the user</code>	
<code>return 0;</code>	Signal successful completion
<code>}</code>	End of main

WARNING — Always Check for NULL After fopen()

Calling `fprintf(NULL, ...)` or `fgets(buffer, size, NULL)` will crash your program with a segmentation fault. There is no recovery from a NULL file pointer except to exit cleanly. The NULL check after every `fopen()` is not optional defensive coding — it is the minimum viable error handling. In production code, you would also log the error and show a user-friendly message. Start that habit now.

03 Reading from a File

Reading uses the same open-operate-close pattern, but with mode `"r"` and `fgets()` or `fscanf()` instead of `fprintf()`. `fgets(buffer, size, fp)` reads one line at a time (up to `size-1` characters), stores it in the buffer, and returns NULL when it reaches the end of the file — making it perfect as a while loop condition. `fscanf()` works like `scanf` but reads from the file; use it for structured data like numbers in a specific format.

<code>#include <stdio.h></code>	Standard I/O
<code>int main() {</code>	Program entry point
<code>FILE *fp;</code>	FILE pointer
<code>char line[200];</code>	Buffer to hold one line at a time — size larger than longest expected line
<code>fp = fopen("daily_goals.txt", "r");</code>	Open for reading — returns NULL if file does not exist
<code>if (fp == NULL) {</code>	Handle file-not-found
<code>printf("No goals saved yet. Add some first!\n");</code>	User-friendly message
<code>return 1;</code>	Exit with error code
<code>}</code>	File found and open
<code>printf("--- Your Daily Goals ---\n");</code>	Header before printing goals
<code>while (fgets(line, 200, fp) != NULL) {</code>	<code>fgets</code> returns NULL at end-of-file — loop exits automatically
<code>printf("%s", line);</code>	Print each line — <code>fgets</code> keeps the <code>\n</code> so no extra <code>\n</code> needed
<code>}</code>	All lines read
<code>fclose(fp);</code>	Close the file — always
<code>return 0;</code>	Signal successful completion
<code>}</code>	End of main

PRO TIP — `fgets` vs `fscanf` for Reading

`fgets` reads an entire line as a string (including spaces) and is the safe, general-purpose choice. `fscanf` reads formatted data (like `%d` or `%f`) and stops at whitespace, making it ideal when you know the file has one number or word per line with no variation. For reading structured data like `hours[i]` saved as one float per line, `fscanf(fp, "%f", &hours[i])` is cleaner. For reading free-form text lines (like goal statements), `fgets` is the right tool.

TRY THIS LIVE

Combine both programs into one menu-driven application using a do-while loop: option 1 adds a new goal (open in "a" append mode, write one line, close), option 2 displays all goals (open in "r" mode, read and print all lines, close), option 3 exits. Test that adding goals accumulates them across multiple runs — this is real data persistence. Notice how appending with "a" versus overwriting with "w" produces very different results.

CASE STUDY — Tally Accounting Software: File I/O Powering Gujarat's MSMEs

Context — Tally Solutions (Bengaluru) produces Tally.ERP, used by over 70% of Indian small and medium enterprises for accounting. In Gujarat — one of India's most MSME-dense states (textile traders in Surat, diamond merchants in Surat and Ahmedabad, chemical firms in Ankleshwar) — Tally is virtually the universal accounting standard.

Challenge — Tally must save every voucher, ledger entry, and invoice between sessions — and retrieve

them instantly when the user reopens a company file. Data that cannot be persisted reliably is worthless for accounting. Tally cannot use a database server because most SME installations are single-computer offline setups.

What they did — Tally uses its own proprietary file-based storage format (.dat files) built on sequential C-style file I/O: structured records are written to binary and text files using file write operations, and loaded back on startup using file read operations. The core pattern — fopen, write-in-loop, fclose on save; fopen, read-in-loop, fclose on load — is exactly what you practised today.

Result — Tally processes accounting data for 70 lakh+ company installations across India, with every rupee of every transaction persisted through file I/O. For Gujarat's textile, diamond, and pharma MSMEs, the Rs 25-lakh-crore transaction data they trust to Tally lives in files written exactly like your `daily_goals.txt`.

Takeaway — The fopen / fprintf / fclose cycle you wrote today is the same pattern that India's most widely used accounting software uses to store crores of rupees of business data.

Source: Tally Solutions press releases, tallysolutions.com; MSME Ministry data on Gujarat MSME ecosystem, 2023

“A file is RAM that survives — everything worth keeping must eventually be written to one.”

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Problem Solving through Programming in C — Lecture 27: File Handling — fopen, fprintf, fgets (IIT Kharagpur, Prof. Anupam Basu)

Link — <https://nptel.ac.in/courses/106105085>

WHERE TO GO NEXT

Your programs can now save and load data between sessions. This is the last major primitive you need for the mini project — which combines loops, arrays, functions, and file handling into one complete application.

Explore deeper — Search 'binary file I/O in C: fwrite and fread' — instead of writing text line by line, you can write an entire struct array to a file as raw bytes in one call. Much faster for large datasets.

Try on your own — Extend your goals program to number each goal automatically as it is saved. On loading, display the goals with their numbers. Then add an option to delete a specific goal by rewriting the file without that line.

Connect the dots — Next session: debugging and best practices — you will learn to read GCC error messages, comment your code professionally, and apply naming conventions. Then comes the mini project.

Going further — Research 'CSV file reading in C' — how to parse a comma-separated file line by line using `strtok()` and `sscanf()` to extract structured fields. This is how real-world C programs import data from spreadsheets.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a C programming teacher. Explain file handling in C in simple Hinglish. Use the analogy of a dukaan owner who writes daily accounts in a register (file write) and reads them the next morning to check yesterday's totals (file read). Explain what happens if they forget to close the register properly (not calling fclose). Keep it under 150 words.

SESSION

14

60 MIN · FUNDAMENTALS

Debugging & Best Practices

◆ Learning Objectives

- Classify any C error as syntax, runtime, or logical — and know the right strategy to fix each
- Read a GCC error message and identify the file name, line number, and error type
- Apply the top-down, one-at-a-time fix strategy to clear a list of compiler errors efficiently
- Write meaningful comments that explain the why, not just the what
- Apply consistent indentation, naming, and spacing conventions used in industry C code

01 The Three Types of C Errors

Every bug you will ever encounter in C falls into exactly one of three categories. Syntax errors: the code violates grammar rules and the compiler refuses to produce an executable — detected at compile time. Runtime errors: the code compiles but crashes or behaves abnormally while running — triggered by specific inputs or conditions. Logical errors: the code compiles, runs, and produces output — but the output is wrong. Logical errors are the hardest to find because C gives you no error message at all.

Error Type	When Detected	GCC Output	Common Examples	Fix Strategy
Syntax	Compile time	error: ... on line N	Missing ;, unmatched {, undeclared variable	Read GCC message -> go to that line -> fix -> recompile
Runtime	During execution	Program crashes or hangs	Division by zero, NULL pointer deref, array out of bounds	Add input validation and NULL checks
Logical	Never — must test	No error — wrong output	Wrong formula, off-by-one, = instead of ==	Add printf statements to trace values; desk-check logic

CONCEPT — Why Logical Errors Are the Most Dangerous

A syntax error stops the build. A runtime error crashes the program visibly. A logical error silently produces wrong results — and may do so for months before anyone notices. Example: calculating tax as $\text{amount} * 0.18$ instead of $\text{amount} * 18 / 100$ (which are identical) but then using $\text{amount} * 1.8$ by a single digit typo gives 10x the correct tax. The program runs perfectly; the output is catastrophically wrong. The only defence is testing with known expected values.

02 Reading GCC Error Messages

A GCC error message always follows the same format: filename.c:line:column: error: description. The filename tells you which file has the problem. The line number tells you exactly where to look. The column number points to the character position. The description — though sometimes cryptic — names the specific violation. Reading this information precisely, rather than panicking at a wall of red text, is the most important debugging skill a beginner can develop.

```
// Buggy code:
int main() {
    int x = 10
    int y = 0;
    printf("%d\n", x / y);
    if (x = 5) printf("five\n");
}
```

--- This code has three bugs ---

Line 1

Line 2 — Bug 1: missing semicolon

Line 3

Line 4 — Bug 2: division by zero (runtime)

Line 5 — Bug 3: = (assignment) instead of == (comparison) — logical error

Line 6

--- What GCC prints ---

```
// GCC output for Bug 1:
// hello.c:3:5: error: expected ';' before 'int'
File: hello.c | Line 3 | Column 5 | Description: GCC saw 'int' but expected ';' first
```

--- Fix Bug 1: add ; to end of line 2. ---

Recompile after each fix — do not try to fix all at once

```
// Bug 2 (runtime) is caught only when y == 0 at runtime.
Fix: add if (y != 0) guard before dividing
```

```
// Bug 3 (logical): x = 5 assigns 5 to x (always truthy).
Fix: change = to == inside the if condition
```

PRO TIP — Fix Top-Down, One at a Time

When GCC lists 8 errors, do not try to fix them simultaneously. Fix only the first error in the list — the one on the lowest line number — then recompile immediately. One real error near the top often causes 5-6 false cascade errors below it. After a single fix, those false errors usually vanish. Developers who fix top-down resolve error lists 3-4 times faster than those who try to address everything at once.

03 Commenting Best Practices

Comments explain the why and the what-is-not-obvious — not the what-you-can-already-read. A comment that says 'add 1 to i' above `i++` adds noise, not knowledge. A comment that says 'offset by 1 because display is 1-indexed but array is 0-indexed' above `i+1` adds genuine context that saves future readers from confusion. Aim for fewer, better comments over many obvious ones — a well-named variable or function often eliminates the need for a comment entirely.

```
// BAD COMMENTS — state what the code already shows:
--- Avoid this style ---
i++;           // increment i by 1           Redundant — the code is self-explanatory
total = 0;     // set total to zero          Redundant

// GOOD COMMENTS — explain context, intent, and non-obvious choices:
--- Use this style ---
// Start from index 1 — index 0 is reserved for the module header
// Explains a non-obvious design decision
average = (float)total / n; // cast needed — both are int by default
// Explains the why of the cast

/*                                     Block comment for function headers:
 * calculateAverage: computes mean of float array
 * What it does
 * Parameters: arr[] - input array, n - number of elements
 * What goes in
 * Returns: float average value          What comes out
 */                                     End of block comment — use this above every function
```

04 Naming Conventions and Style

Consistent style makes code readable, reviewable, and maintainable. In C, the widely accepted convention is snake_case for variable and function names (student_marks, calculate_average), UPPER_CASE for constants defined with #define (MAX_SIZE, PASSING_MARKS), and descriptive names that reveal intent. Indentation is always 4 spaces per level — never mix tabs and spaces, as they render differently across editors. These conventions are not personal preference; they are professional standards that every team enforces.

Category	Bad — Avoid	Good — Use	Reason
Variables	x, a, temp2, sm	student_marks, screen_time	Name reveals purpose — no guessing
Functions	calc(), dolt(), f1()	calculateAverage(), saveFile()	Verb + noun describes the action
Constants	passmark, MaxSize	PASS_MARK, MAX_SIZE	UPPER_CASE signals immutability
Loop vars	i in a function, x	i, j, k (loops only)	Single letters acceptable only for loop counters
Booleans	flag, status, val	isValid, hasData, isRunning	is/has prefix clarifies true/false meaning

PRO TIP — Let Your Editor Help

VS Code with the C/C++ extension auto-formats your code on save (Shift+Alt+F on Windows). It fixes indentation, spacing around operators, and brace placement automatically. Enable format-on-save in VS Code settings: Editor: Format On Save -> true. This removes the manual effort of style enforcement and means your code always looks professional. Pair it with gcc -Wall to catch warnings, and you have a lightweight but powerful quality check built into your daily workflow.

CASE STUDY — Infosys ITR Portal: When All Three Error Types Compound at Launch

Context — Infosys was contracted by India's Income Tax Department to build the new e-filing portal (incometax.gov.in), launched in June 2021. The portal was meant to serve crores of Indian taxpayers — including lakhs of Gujarat's businesses and professionals — for filing returns before the deadline.

Challenge — The portal crashed on launch day itself. Millions of users including chartered accountants and businesses across Gujarat reported login failures, form submission errors, and incorrect tax calculation results simultaneously.

What they did — Infosys's debugging team categorised faults into the same three types: syntax/validation errors (login form fields not accepting valid inputs — quickly patched), runtime errors (null pointer exceptions when certain form combinations were submitted — required edge-case handling), and logical errors (tax computation formulas producing incorrect results for certain income slabs — took weeks to identify and fix). Fixes were deployed in order: validation first (fastest), runtime handling second, logic verification last.

Result — After several months of patches, the portal stabilised. The Finance Ministry publicly acknowledged the issues. The incident became a widely cited example of how all three error categories, when deployed simultaneously at national scale without adequate testing, compound into a system-wide failure.

Takeaway — Every bug you encounter in this module fits one of the three categories. Classifying it correctly — syntax, runtime, or logical — determines the fix strategy. The ITR portal failure happened because this classification and the corresponding testing were not thorough enough before launch.

Source: Finance Ministry Statement on ITR Portal Issues, June 2021; Infosys Press Release, infosys.com; ICAI communications to members, 2021

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Problem Solving through Programming in C — Lecture 29: Debugging Techniques and Code Quality (IIT Kharagpur, Prof. Anupam Basu)

Link — <https://nptel.ac.in/courses/106105085>

WHERE TO GO NEXT

You now have both the technical skills (Sessions 01–13) and the professional habits (Session 14) needed to build real software. The mini project is next — the first time everything comes together.

Explore deeper — Search 'GDB debugger C tutorial' — GDB is the GNU Debugger that lets you pause a

running C program, inspect variable values at any line, and step through code one instruction at a time. It is the professional alternative to printf debugging.

Try on your own — Take any program you wrote in Sessions 09–13 and perform a full code review on yourself: classify every comment as good or redundant, rename every variable that does not clearly reveal its purpose, verify every array loop uses $i < \text{size}$ and not $i \leq \text{size}$, and check every fopen call has a NULL guard.

Connect the dots — Next session: Mini Project — Student Productivity Tracker. You will combine loops, arrays, functions, and file handling into one complete, menu-driven application.

Going further — Look up 'Valgrind for C memory error detection' — it catches memory leaks, invalid pointer dereferences, and buffer overflows at runtime. Used by professional C developers worldwide to find bugs that GCC's compiler warnings cannot catch.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a C programming teacher. Explain the three types of C errors — syntax, runtime, and logical — in simple Hinglish. Use the analogy of writing a job application: a syntax error is a spelling mistake that the HR rejects immediately, a runtime error is when you show up for the interview but the address was wrong, and a logical error is when you attend the interview and say the wrong salary expectation. Keep it under 150 words.

SESSION

15

120 MIN · MINI PROJECT

Mini Project: Student Productivity Tracker

◆ Learning Objectives

- Design a complete menu-driven console application from a plain-language brief
- Decompose the application into named functions — one function, one responsibility
- Use a float array to store daily study hours across a 7-day week
- Persist and reload the weekly data using file I/O across program sessions
- Generate a formatted weekly progress report with a text-based bar chart

01 Project Brief

Build a console application called Student Productivity Tracker. The program lets a student log their daily study hours for each day of the week, view a weekly summary with a simple bar chart, and save their data so it persists between program runs. Every feature from Sessions 06 through 13 is required: a do-while menu (Session 06), a float array for storage (Session 09), string labels for days (Session 10), functions for each operation (Session 08), and file I/O for persistence (Session 13).

CONCEPT — Application Architecture

The app has five responsibilities, each mapped to one function: `logHours()` — records hours for a specific day. `displayReport()` — prints the weekly summary with bar chart. `calculateAverage()` — computes the average hours per day. `saveToFile()` — writes the hours array to disk. `loadFromFile()` — reads it back on startup. `main()` only contains the do-while menu and calls these functions — it does no computation itself. This single-responsibility design is the same principle used in every professional codebase.

02 Data Design and Program Constants

Before writing a single function, define the data structure and constants. The core data is a float array of 7 elements — one per day. The filename for persistence is a string constant defined once at the top so changing it is a one-line update. Day names are stored in a char pointer array for clean display output. Defining these once as constants means they never appear as magic numbers or hardcoded strings buried inside functions.

```
#include <stdio.h>
#include <string.h>
```

```
#define DAYS 7
```

```
#define FILENAME "study_log.txt"
```

Standard I/O — `printf`, `scanf`, FILE operations

For string functions used in display

One constant controls the entire program's week size

One constant for the data file — change here to rename everywhere

```
// Function prototypes – declared before main()
```

Compiler needs to know signatures before they are called

```
void logHours(float hours[], int day, float value);
```

Log hours for one day

```
float calculateAverage(float hours[], int n);
```

Return average of n float values

```
void displayReport(float hours[], int n);
```

Print weekly summary with bar chart

```
void saveToFile(float hours[], int n);
```

Write array to FILENAME

```
void loadFromFile(float hours[], int n);
```

Read array from FILENAME on startup

03 main() — Menu and Control Flow

main() should be the shortest function in the program. Its only jobs are: initialise the array, load saved data, display the menu in a do-while loop, and delegate every action to the appropriate function. If main() is doing computation — calculating averages, writing to files directly — that logic belongs in a named function. A well-structured main() reads like a table of contents for the program.

```
int main() {
    float hours[DAYS] = {0};
    int choice, day;
    float value;
    loadFromFile(hours, DAYS);
    do {
        printf("\n=== Study Tracker ===\n");
        printf("1. Log today's hours\n");
        printf("2. View weekly report\n");
        printf("3. Save and exit\n");
        printf("Choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1:
                printf("Day (1-7): "); scanf("%d", &day);
                Read day number
                printf("Hours studied: "); scanf("%f", &value);
                Read hours
                logHours(hours, day - 1, value);
                day-1 converts 1-7 display to 0-6 index
                break;
            case 2: displayReport(hours, DAYS); break;
                Show report
            case 3: saveToFile(hours, DAYS); printf("Saved.\n"); break;
                Save and signal exit
            default: printf("Invalid. Enter 1, 2, or 3.\n");
                Reject bad input
        }
    } while (choice != 3);
}
```

Entry point

Initialise all 7 slots to 0.0 — {0} fills the rest with zeros

Menu choice and day selector

Hours to log

Load any previously saved data at startup

do-while: menu always shows at least once

Menu header

Option 1

Option 2

Option 3

Prompt

Read menu selection

Dispatch to the right function

Log hours branch

Exit switch

```

    }
    } while (choice != 3);
    return 0;
}

```

End switch
 Loop until user chooses 3
 Signal successful completion
 End of main

04 Core Functions

Each function is self-contained and testable in isolation. `logHours` validates the day index before writing to prevent out-of-bounds access. `displayReport` loops through all days and prints a text bar chart using a nested loop — one pipe character per hour of study. `calculateAverage` is a pure function with no side effects: it takes the array and returns a float. This clean separation means you can test each piece independently before integrating.

```

void logHours(float hours[], int day, float value) {
    day is already converted to 0-based index by caller
    if (day >= 0 && day < DAYS) {
        hours[day] = value;
    } else {
    }
    printf("Day %d logged: %.1f hrs\n", day + 1, value);
    printf("Invalid day. Enter 1 to 7.\n");
}

```

Bounds check — never trust raw user input as an array index
 Write to the array slot
 Confirm with 1-based display
 Out-of-range day
 User-friendly error — no crash
 End logHours

```

float calculateAverage(float hours[], int n) {
    float total = 0; int i;
    for (i = 0; i < n; i++) total += hours[i];
    return total / n;
}

```

Pure computation — no printf inside
 Accumulator and counter
 Sum all elements
 Return the mean

```

void displayReport(float hours[], int n) {
    char *days[] = {"Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"};
    int i, j;
    printf("\n--- Weekly Report ---\n");
    for (i = 0; i < n; i++) {
    }
    printf("%s %4.1f hrs ", days[i], hours[i]);
}

```

Prints the weekly summary with bar chart
 Day labels — char pointer array
 Outer (days) and inner (bar chart) counters
 Report header
 One line per day
 Day name and hours, right-aligned to 4.1 width

```

        for (j = 0; j < (int)hours[i]; j++)
            printf("|");
        printf("\n");
    }
    printf("Average: %.2f hrs/day\n", calculateAverage(hours, n));
    Call calculateAverage — reuse, don't rewrite
}

```

Inner loop: one | per hour studied
Visual bar — 3 hrs = |||, 7 hrs = |||||
Newline after each day's bar
All 7 days displayed
End displayReport

05 File Persistence

saveToFile writes one float per line using fprintf. loadFromFile uses fscanf in a loop to read them back in the same order. loadFromFile silently does nothing if the file does not exist yet — this is the correct behaviour on first run, not an error. saveToFile is called only when the user selects option 3, so partial sessions are not accidentally saved with zeros overwriting real data.

```

void saveToFile(float hours[], int n) {
    FILE *fp = fopen(FILENAME, "w");
    if (fp == NULL) { printf("Save failed.\n"); return; }
    int i;
    for (i = 0; i < n; i++) fprintf(fp, "%.1f\n", hours[i]);
    fclose(fp);
}

void loadFromFile(float hours[], int n) {
    FILE *fp = fopen(FILENAME, "r");
    if (fp == NULL) return;
    int i;
    for (i = 0; i < n; i++) fscanf(fp, "%f", &hours[i]);
    fclose(fp);
    printf("Previous data loaded.\n");
}

```

Write array to disk
Open for write — overwrites previous save
Handle fopen failure gracefully — NULL check
Loop counter
One float per line — %.1f matches fscanf %f on read
Flush and close — never skip
Read array from disk
Open for reading
No saved file yet — first run. Silent return is correct here.
Loop counter
Read each float back into the array
Close file
Confirm to user that saved data is available

EXTENSION CHALLENGES — For Students Who Finish Early

Level 1: Add a 'Reset week' option that sets all hours to 0.0 and saves immediately. Level 2: In displayReport(), flag any day with 0.0 hours in red text using ANSI escape codes (printf("\033[31m0.0\033[0m")). Level 3: Add a target hours constant (#define TARGET 6.0) and print a motivational message if the weekly average is below target. Level 4: Store multiple weeks by appending week

number to the filename (study_log_w1.txt, study_log_w2.txt) and add a 'View past week' menu option.

WHERE TO GO NEXT

You have built a complete, persistent, menu-driven C application from scratch. Every concept from this module — variables, operators, conditions, loops, patterns, functions, arrays, strings, pointers, structures, file handling, and debugging — was in play. That is Module 3 complete.

Explore deeper — Review your complete project code against the Session 14 style checklist: every function has a block comment, every variable name is descriptive, all array loops use $i < \text{size}$, and every fopen has a NULL check. Clean code is the first professional habit.

Try on your own — Share your tracker with a classmate and have them try to break it: enter negative hours, enter a day number of 0 or 8, enter letters instead of a number. Fix every crash they trigger — this is manual testing and it is how real software is hardened.

Connect the dots — Module 4 begins with OOP — Object-Oriented Programming. The struct Student you built in Session 12 and the functions that operated on it are the procedural version of what C++ classes formalise. Every concept carries forward.

Going further — Extend the tracker into a full semester planner: add a subjects array (string array of 5 subject names), log hours per subject per day (a 2D array float hours[7][5]), and generate a subject-wise weekly report. This is a natural extension that uses every session's content at once.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a C programming mentor reviewing a student's mini project. The student has built a Student Productivity Tracker in C with functions for logging, reporting, saving, and loading. Give them three specific, actionable improvements they can make — one for code structure, one for user experience, and one for robustness (error handling). Respond in simple Hinglish. Keep it under 200 words.

REVIEW



INTERVIEW PREP · MODULE 3

Interview Questions

◆ SE – Introduction to Programming (C Language)

Rehearse each answer out loud, aiming for about 60 seconds. For code questions, explain your logic step-by-step — interviewers care as much about your thinking as the syntax.

1

What is a program, and what is a programming language?

One is a set of instructions, the other is the tool to write them.

MODEL ANSWER

A program is a set of instructions that a computer executes to perform a specific task, such as calculating a result or displaying information. A programming language is the formal syntax used to write those instructions in a way both humans can read and machines can process. Languages like C, Python, and Java each have their own rules and are suited to different types of tasks. The source code written in a programming language is then translated into machine code that the CPU actually runs.

2

Explain the difference between a compiler and an interpreter.

Think about how and when each translates source code.

MODEL ANSWER

A compiler translates the entire source code into machine code all at once before the program runs, producing an executable file — C uses this approach. An interpreter translates and executes the code line-by-line at runtime, without creating a separate executable — Python works this way. Compiled programs generally run faster because translation is done ahead of time, while interpreted languages offer more flexibility for interactive testing. Some modern languages (like Java) use a hybrid approach with bytecode and a virtual machine.

3

Describe the basic structure of a C program.

Include the header, main function, and return statement.

MODEL ANSWER

A C program starts with preprocessor directives like `#include <stdio.h>`, which import standard libraries needed for functions like `printf`. The program execution begins at the `main()` function, defined as `int main()`. Inside `main()`, you write your statements — variable declarations, logic, and output calls. The function ends with `return 0;`, which tells the OS that the program completed successfully. Without this structure, the C compiler will throw errors.

4

List four basic data types in C and give a short example of each.

Think of whole numbers, decimals, characters, and larger decimals.

MODEL ANSWER

The four basic data types are: `int` for whole numbers (e.g., `int age = 20;`), `float` for single-precision decimal numbers (e.g., `float price = 99.5;`), `char` for a single character stored as its ASCII value (e.g., `char grade = 'A';`), and `double` for double-precision decimals with greater accuracy (e.g., `double pi = 3.14159265;`). Choosing the right data type matters for both memory efficiency and avoiding precision errors.

5

What are the rules for naming variables in C?

Consider allowed characters, starting characters, and reserved words.

MODEL ANSWER

Variable names in C can contain letters, digits, and underscores but must begin with a letter or underscore — not a digit. Names are case-sensitive, so `score` and `Score` are different variables. Reserved keywords like `int`, `return`, and `while` cannot be used as variable names. It is good practice to use descriptive names like `studentAge` rather than single letters, which makes code easier to read and maintain.

6

What is the difference between arithmetic, logical, and relational operators in C?

Think about what each operator type is used to compare or calculate.

MODEL ANSWER

Arithmetic operators perform mathematical calculations: `+`, `-`, `*`, `/`, and `%` (modulus). Relational operators compare two values and return true (1) or false (0): `==`, `!=`, `<`, `>`, `<=`, `>=`. Logical operators combine multiple conditions: `&&` (AND is true if both sides are true), `||` (OR is true if either side is true), and `!` (NOT flips the result). Understanding when to use each type is essential for writing correct conditions in if-statements and loops.

7

Explain the difference between if, else if, and switch-case statements in C.

MODEL ANSWER

An `if` statement tests a single condition and executes a block if it is true. `else if` adds additional conditions to check in sequence when the previous `if` was false, forming a ladder of checks. `switch-case` is used when a single variable needs to be compared against multiple fixed values — it is cleaner than a long `if-else if` chain for cases like menu options. Switch works with integer and char types; if none of the cases match, the `default` block runs.

8

What is the main difference between entry-controlled and exit-controlled loops in C?

Think about when the loop condition is checked.

MODEL ANSWER

In entry-controlled loops (`for` and `while`), the condition is checked before the loop body executes — if the condition is false from the start, the body never runs. In an exit-controlled loop (`do-while`), the body executes at least once before the condition is checked at the end. A `do-while` loop is therefore useful when you need to run the code at least once regardless of the condition, such as displaying a menu and then asking the user to choose an option.

9

Write a C program using a for loop to print numbers from 10 down to 1.

MODEL ANSWER

The program would be: `#include <stdio.h> int main() { for(int i = 10; i >= 1; i--) { printf("%d\n", i); } return 0; }`. The loop initialises `i` at 10, continues while `i` is greater than or equal to 1, and decrements `i` by 1 after each iteration. The `printf` statement prints the current value followed by a newline. This pattern — counting down by decrementing the loop variable — is a common interview question.

10

What is a function in C, and why do we use them?

Think about code reuse and organization.

MODEL ANSWER

A function is a named block of code that performs a specific task and can be called multiple times from different parts of the program. Functions promote code reuse — instead of writing the same logic repeatedly, you write it once and call the function wherever needed. They also make code easier to read, test, and debug by breaking a large program into smaller, focused pieces. In C, every function has a return type, a name, a parameter list, and a body.

11

Explain the difference between pass-by-value and pass-by-reference in C.

MODEL ANSWER

In pass-by-value, the function receives a copy of the variable's value — any changes made inside the function do not affect the original variable in the caller. In pass-by-reference, the function receives a pointer to the original variable's memory address — changes made through the pointer directly modify the original value. Pass-by-reference is used when a function needs to modify the caller's variable, such as a `swap()` function that exchanges two values. All basic type arguments in C are passed by value by default.

12

Write a C function to swap two numbers using pointers.

MODEL ANSWER

The function signature is `void swap(int *a, int *b)`. Inside: declare a temp variable `int temp = *a;`, then `*a = *b;` and `*b = temp;`. The asterisk (`*`) dereferences the pointer to access the actual value at that memory address. When calling: `swap(&x, &y);` — the ampersand (`&`) passes the address of `x` and `y`. Without pointers, the swap would only affect local copies inside the function and the original variables would remain unchanged.

13

What is an array in C, and how is a 1D array different from a 2D array?

MODEL ANSWER

An array is a collection of elements of the same data type stored in contiguous memory locations, accessed using an index starting from 0. A 1D array is a single list of elements, like `int scores[7];` for 7 days of data. A 2D array is a table or matrix, like `int marks[5][3];` for 5 students and 3 subjects. Accessing a 1D element uses one index (`scores[2]`), while a 2D element needs two (`marks[1][2]` for row 1, column 2).

14

What is a pointer in C, and how is it different from a normal variable?*Think about memory addresses.***MODEL ANSWER**

A normal variable stores a data value (like an integer or character) directly. A pointer is a variable that stores the memory address of another variable — it 'points to' where the data is stored. You declare a pointer with `int *p;` and assign it an address with `p = &x;`. You access the value at that address using dereference: `*p`. Pointers are powerful for dynamic memory, arrays, and passing large data structures to functions efficiently.

15

How do you declare and initialize a structure in C?*Think about grouping different data types together.***MODEL ANSWER**

A structure is declared using the `struct` keyword followed by a name and a block of member variables. For example: `struct Student { char name[50]; int marks; char grade; };`. To create an instance: `struct Student s1;` and then assign values like `s1.marks = 85;`. You can also initialize at declaration: `struct Student s1 = {"Riya", 85, 'A'};`. Structures let you group related data of different types under one name, which is useful for representing real-world entities like students, employees, or products.

16

Explain the difference between 'r', 'w', and 'a' modes when opening a file in C.**MODEL ANSWER**

Mode `r` opens an existing file for reading only — the program cannot write to it, and if the file doesn't exist, `fopen` returns NULL. Mode `w` opens a file for writing — it creates the file if it doesn't exist, or erases all existing content if it does, so use it carefully. Mode `a` opens a file for appending — it creates the file if needed, but adds new data at the end without erasing what's already there. For logging daily records, `a` mode is typically the right choice.

17

What is the difference between a syntax error and a runtime error in C?*Think about when the error is detected.***MODEL ANSWER**

A syntax error is a mistake in the code that violates the rules of the C language — such as a missing semicolon or mismatched brackets. The compiler catches these before the program runs and refuses to produce an executable. A runtime error happens while the program is executing — for example, dividing by zero, accessing an array out of bounds, or dereferencing a NULL pointer. Runtime errors are harder to catch because the code compiles successfully but crashes or behaves incorrectly during execution.

18

Suppose your code compiles but the output is not what you expect. How would you start debugging it?

Think about checking logic, print statements, and input values.

MODEL ANSWER

Start by adding ``printf`` statements at key points in the code to print variable values and confirm which code paths are being executed. Check your loop conditions and any conditional logic — a common mistake is an off-by-one error or a wrong operator. Trace through the logic manually with a small example to compare expected versus actual values. For more complex issues, use a debugger like GDB to step through the program line by line, inspect variable values, and set breakpoints.

19

How would you combine arrays, loops, and file handling to log daily study hours and generate a weekly progress report?

MODEL ANSWER

Declare an array ``float hours[7];`` and use a ``for`` loop to ask the user to input each day's study hours. Open a file with mode ``a`` and write each entry using ``fprintf``. At the end of the week, read the file back with ``fscanf`` or re-use the array to calculate the total and average using a loop with a sum variable. Print the weekly summary to the console and optionally write it to a separate report file. This is the core pattern of the Student Productivity Tracker mini project.

20

How would you design a menu-driven console application for tracking student productivity?

MODEL ANSWER

Display a menu with options (1 = Log hours, 2 = View weekly report, 3 = Exit) using a ``do-while`` loop so the menu re-appears after each action. Use a ``switch-case`` on the user's choice to call the appropriate function. The log function appends a record (date, hours) to a file. The report function reads the file, sums the hours, and calculates averages. Wrap file operations in error checks (if ``fopen`` returns NULL, inform the user). This structure is clean, testable, and easy to extend.